

Control de Concurrencia y Database Recovery

SEMANA 06

Percy Lovon
plovon@utec.edu.pe

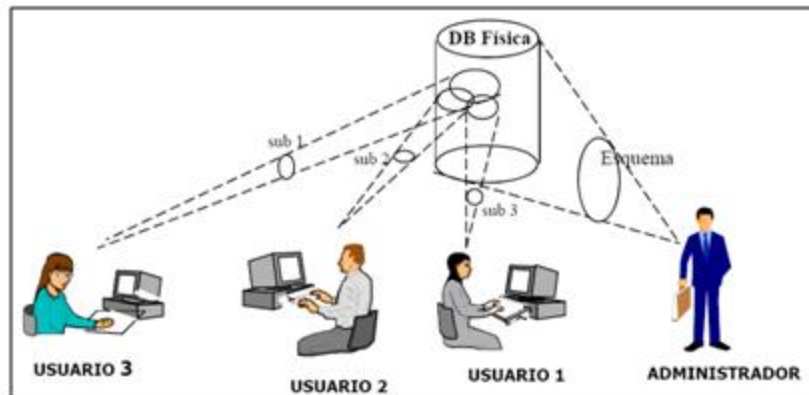
1.



Control de Concurrencia

*Database Management Systems 3rd Edition, Ramakrishnan,
Chapter 16-17*

Concurrencia



Transaction 1

```
SQL> UPDATE employees
2 SET salary=salary+100
3 WHERE employee_id=100;
```

Transaction 2

```
SQL> UPDATE employees
2 SET salary=salary*1.1
3 WHERE employee_id=100;
```



¿Qué es concurrencia?

- ✓ Coordinación de la ejecución simultanea de transacciones en un SGBD que soporta **multiprocesamiento**.
 - Aumenta la productividad por la mejor utilización de recursos y produce que el tiempo de los usuarios se reduzca en promedio
- ❖ Cuando se ejecutan transacciones de manera concurrente puede haber interferencias entre ellas que produzcan resultados no correctos.
 - Una transacción individual producirá un estado correcto de la BD si se ejecuta de forma **aislada**.

¿Qué es una transacción?

- Un conjunto de operaciones de acceso a base de datos que conforman una unida lógica de trabajo.
- El SGBD debe garantizar que se realicen adecuadamente a pesar de la existencia de fallos.
- Puede iniciar con una instrucción BEGIN TRANSACTION (depende del SGBD).
- Hay dos operaciones obligatorias:
 - COMMIT: transacción exitosa, los cambios se deben hacer efectivos (persistencia)
 - ROLLBACK: transacción no exitosa, los cambios se deben deshacer.

Propiedades de una transacción: ACID

Atómica

- Se realizan todas las acciones o no se realiza ninguna

Consistente

- La base de datos satisfacen todas las restricciones después de una transacción.

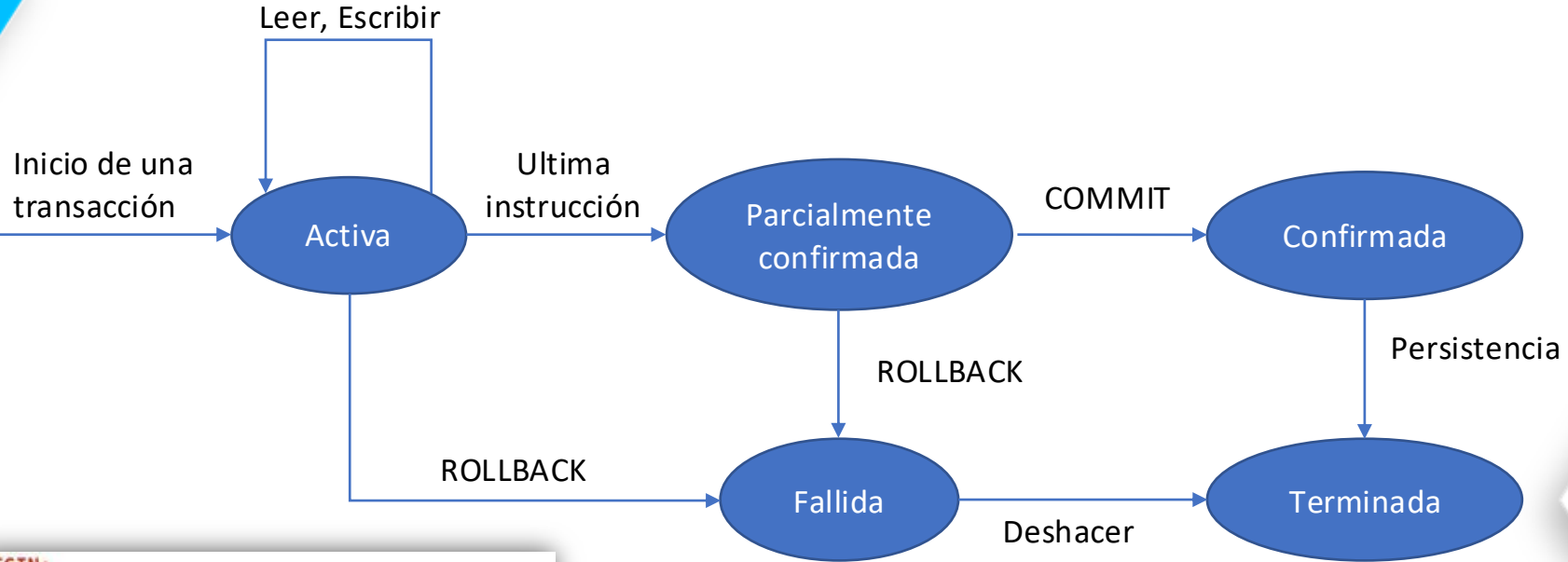
Aislada

- Una transacción no puede afectar otra aunque se ejecuten **concurrentemente**.

Durable

- Una vez que hace COMMIT, la base de datos debe persistir los cambios.

Estados de una transacción



```
BEGIN;  
UPDATE accounts SET balance = balance - 100.00  
WHERE name = 'Alice';  
UPDATE branches SET balance = balance - 100.00  
WHERE name = (SELECT branch_name FROM accounts  
WHERE name = 'Alice');  
UPDATE accounts SET balance = balance + 100.00  
WHERE name = 'Bob';  
UPDATE branches SET balance = balance + 100.00  
WHERE name = (SELECT branch_name FROM accounts  
WHERE name = 'Bob');  
COMMIT;
```

¿Cómo gestionar dos transacciones que acceden a los mismo recursos?

Planificación de Transacciones



Planificación de Transacciones

- Para n transacciones $T1, T2, \dots, Tn$, una planificación:
 - Representa el orden cronológico en el que se ejecutan las instrucciones de dichas transacciones en el sistema.
 - La secuencia debe conservar el orden dentro de cada transacción individual.
- Dadas las siguientes transacciones:

T1
READ(X) $X = X - 50$ WRITE(X) READ(Y) $Y = Y + 50$ WRITE(Y)

T2
READ(X) $TEMP = A * 0.1$ $A = A - TEMP$ WRITE(A) READ(Y) $Y = Y + TEMP$ WRITE(Y)

Planificación de Transacciones

- Se pueden definir las siguientes **planificaciones secuenciales**:

T1	T2
READ(X) X = X - 50 WRITE(X) READ(Y) Y = Y + 50 WRITE(Y)	READ(X) TEMP = A * 0.1 A = A - TEMP WRITE(A) READ(Y) Y = Y + TEMP WRITE(Y)

T1	T2
READ(X) X = X - 50 WRITE(X) READ(Y) Y = Y + 50 WRITE(Y)	READ(X) TEMP = A * 0.1 A = A - TEMP WRITE(A) READ(Y) Y = Y + TEMP WRITE(Y)

- Cada transacción se ejecuta completamente antes de la siguiente.
- Para un conjunto de n transacciones hay $n!$ planificaciones secuenciales.

Planificación de Transacciones

- Pero se pueden definir un numero $m \gg n!$ de planificaciones concurrentes.

T1	T2
READ(X) $X = X - 50$	READ(X) $TEMP = A * 0.1$
WRITE(X) READ(Y)	$A = A - TEMP$ WRITE(A) READ(Y)
$Y = Y + 50$ WRITE(Y)	$Y = Y + TEMP$ WRITE(Y)

Planificación
es
secuenciales

Planificaciones
concurrentes

Problemas de la Concurrency

- **Actualización perdida:** se asume modificación inmediata.

T1	Tiempo	T2	X=100
READ(X)	t1	READ(X)	
	t2		
X=X-10	t3	X=X+100	
	t4		
WRITE(X)	t5	WRITE(X)	
	t6		
COMMIT	t7	COMMIT	
	t8		

Problemas de la Concurrency

- **Actualización perdida:** se asume modificación inmediata.

T1	Tiempo	T2	X
READ(X)	t1	READ(X)	100
	t2		100
X=X-10	t3	X=X+100	
	t4		
WRITE(X)	t5	WRITE(X)	
	t6		
COMMIT	t7	COMMIT	
	t8		

1. Ambas transacciones guardan su copia de X.

Problemas de la Concurrency

- **Actualización perdida:** se asume modificación inmediata.

T1	Tiempo	T2	X
	t1	READ(X)	100
READ(X)	t2		100
	t3	X=X+100	100
X=X-10	t4		100
	t5	WRITE(X)	
WRITE(X)	t6		
	t7	COMMIT	
COMMIT	t8		

1. Ambas transacciones guardan su copia de X.
2. T1 y T2 realizan modificaciones diferentes en X.

Problemas de la Concurrency

- **Actualización perdida:** se asume modificación inmediata.

T1	Tiempo	T2	x
READ(X)	t1	READ(X)	100
	t2		100
X=X-10	t3	X=X+100	100
	t4		100
WRITE(X)	t5	WRITE(X)	200
	t6		
COMMIT	t7	COMMIT	
	t8		

1. Ambas transacciones guardan su copia de X.
2. T1 y T2 realizan modificaciones diferentes en X.
3. T2 actualiza su valor en memoria principal.

Problemas de la Concurrency

- **Actualización perdida:** se asume modificación inmediata.

T1	Tiempo	T2	X
READ(X)	t1	READ(X)	100
	t2		100
	t3	X=X+100	100
X=X-10	t4		100
WRITE(X)	t5	WRITE(X)	200
	t6		90
	t7	COMMIT	90
COMMIT	t8		90

1. Ambas transacciones guardan su copia de X.
2. T1 y T2 realizan modificaciones diferentes en X.
3. T2 actualiza su valor en memoria principal.
4. T1 actualiza y se pierde la actualización de T2.

Problemas de la Concurrency

- **Dependencia no confirmada:** lectura no confirmada.

T1	Tiempo	T2	X=100
READ(X) X=X-10 WRITE(X) COMMIT	t1	READ(X)	
	t2	X=X+100	
	t3	WRITE(X)	
	t4		
	t5		
	t6	ROLLBACK	
	t7		
	t8		

Problemas de la Concurrency

- **Dependencia no confirmada:** lectura no confirmada.

T1	Tiempo	T2	X
READ(X) X=X-10 WRITE(X) COMMIT	t1	READ(X)	100
	t2	X=X+100	100
	t3	WRITE(X)	200
	t4		
	t5		
	t6	ROLLBACK	
	t7		
	t8		

1. T2 actualiza X a 200.

Problemas de la Concurrency

- **Dependencia no confirmada:** lectura no confirmada.

T1	Tiempo	T2	X
	t1	READ(X)	100
	t2	X=X+100	100
	t3	WRITE(X)	200
READ(X)	t4		200
X=X-10	t5		200
	t6	ROLLBACK	
WRITE(X)	t7		
COMMIT	t8		

1. T2 actualiza X a 200.
2. T1 lee el nuevo valor de X.

Problemas de la Concurrency

- **Dependencia no confirmada:** lectura no confirmada.

T1	Tiempo	T2	X
READ(X) X=X-10 WRITE(X) COMMIT	t1	READ(X)	100
	t2	X=X+100	100
	t3	WRITE(X)	200
	t4		200
	t5		200
	t6	ROLLBACK	100
	t7		
	t8		

1. T2 actualiza X a 200.
2. T1 lee el nuevo valor de X.
3. T2 hace rollback por lo que la actualización previa se deshace.

Problemas de la Concurrency

- **Dependencia no confirmada:** lectura no confirmada.

T1	Tiempo	T2	X
READ(X) X=X-10 <u>WRITE(X)</u> COMMIT	t1	READ(X)	100
	t2	X=X+100	100
	t3	WRITE(X)	200
	t4		200
	t5		200
	t6	ROLLBACK	100
	t7		190
	t8		190

1. T2 actualiza X a 200.
2. T1 lee el nuevo valor de X.
3. T2 hace ROLLBACK por lo que la actualización previa se deshace.
4. T1 continua trabajando con el valor actualizado por lo que el nuevo valor de X es **incorrecto**.

Problemas de la Concurrency

Tiempo	Transacción 1 (Transferencia)	Transacción 2 (Genera un Reporte/Retiro)	Valor de X en Memoria	Explicación
t_1	Deposita \$400: <code>write(X = \$500)</code>		\$500	T_1 modifica el dato en memoria, pero no ha hecho COMMIT [cite: 18, 216].
t_2		Lee $X \rightarrow$ obtiene \$500	\$500	¡Lectura no confirmada! T_2 lee el dato modificado por T_1 [cite: 18, 114].
t_3	Ocurre un error o fallo del sistema $\rightarrow$ ROLLBACK		\$100	El sistema deshace el cambio de T_1 [cite: 18, 94]. El valor de X vuelve a \$100 [cite: 94, 282].
t_4		Imprime un estado de cuenta de \$500 o aprueba un retiro de \$400.	\$100	Efecto de la anomalía: T_2 operó sobre datos fantasma/sucios [cite: 18, 114].

Problemas de la Concurrency

- **Suma incorrecta:**

- Si una transacción está calculando una suma sobre varios registros, mientras otras transacciones están actualizando algunos de estos registros, la función suma puede tomar algunos valores antes que sean actualizados y otros después de ser actualizados.

- Ejemplo: compras en líneas .

- **Lectura no repetible:**

- En dos lecturas sucesivas de un elemento, la transacción T_i recibe respuestas distintas.

Los problemas de concurrency generan inconsistencias en la base de datos.

Planificación Serializable

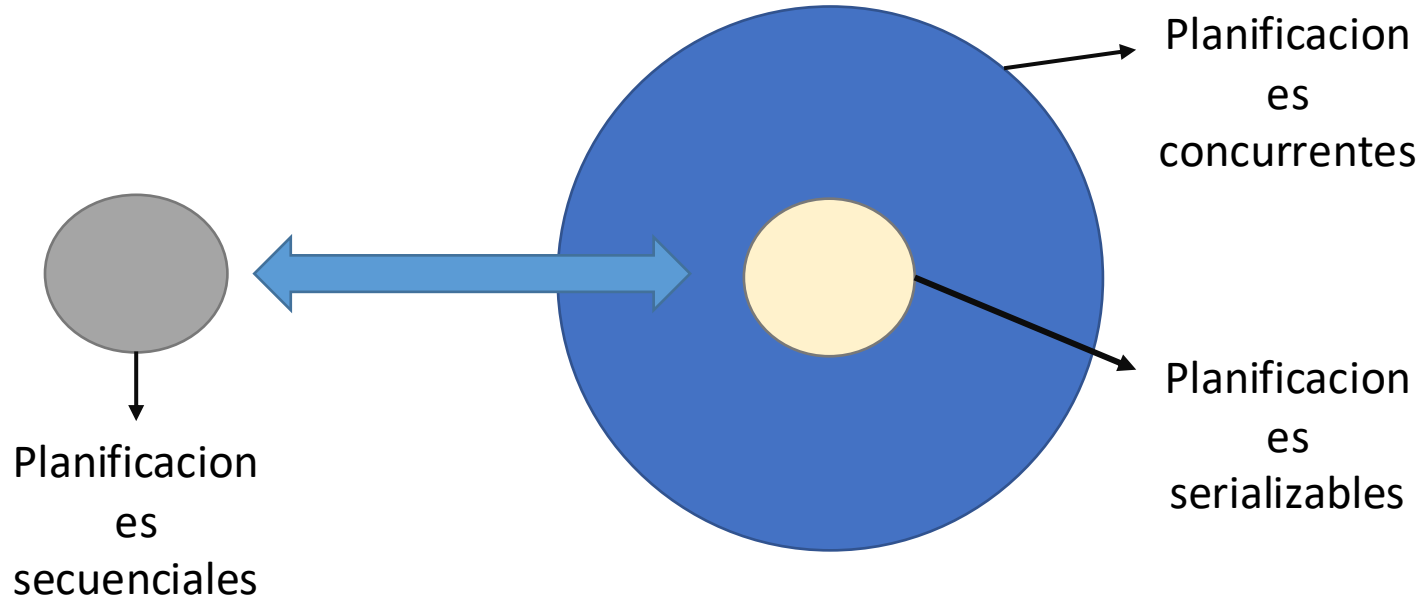
- Una **planificación serializable** (*serializable schedule*) es el **estándar de oro de la corrección** en el procesamiento de transacciones concurrentes .
- Permite obtener lo mejor de ambos mundos: **la alta velocidad del procesamiento intercalado/concurrente** y **la seguridad absoluta de una ejecución en fila/secuencial**
- ¿Cómo asegurar que una planificación concurrente es correcta?
 - Debe tener el mismo efecto que una planificación secuencial.

Una planificación serializable
es una planificación concurrente
equivalente a una secuencial.

A
C
I
D

Planificación Serializable

- Dado el conjunto de todas las planificaciones concurrentes posibles.



Planificación Serializable

- ¿Cómo saber si una planificación es serializable?
 - **Precedencia:**
 - Se tiene que una transacción TA precede a una transacción TB, si TB lee un dato que TA modificó o si TA lee un dato que TB modificará.
 - Se denota como $T_A \rightarrow T_B$
 - **Seriabilidad:**
 - Proceso de determinar que una planificación concurrente es equivalente a una secuencial, es decir si $T_A \rightarrow T_B$ o $T_B \rightarrow T_A$

Planificación Serializable

- Serialización por Conflictos

Se dice que dos instrucciones I_i e I_j (de las transacciones T_i y T_j respectivamente, $i \neq j$) están en conflicto si: son de **distintas transacciones**, operan sobre el **mismo elemento** de datos, y al menos una de ellas **actualiza** dicho dato.

Para probar si un plan es serializable por conflictos se usa un grafo de precedencia o grafo de serialización.

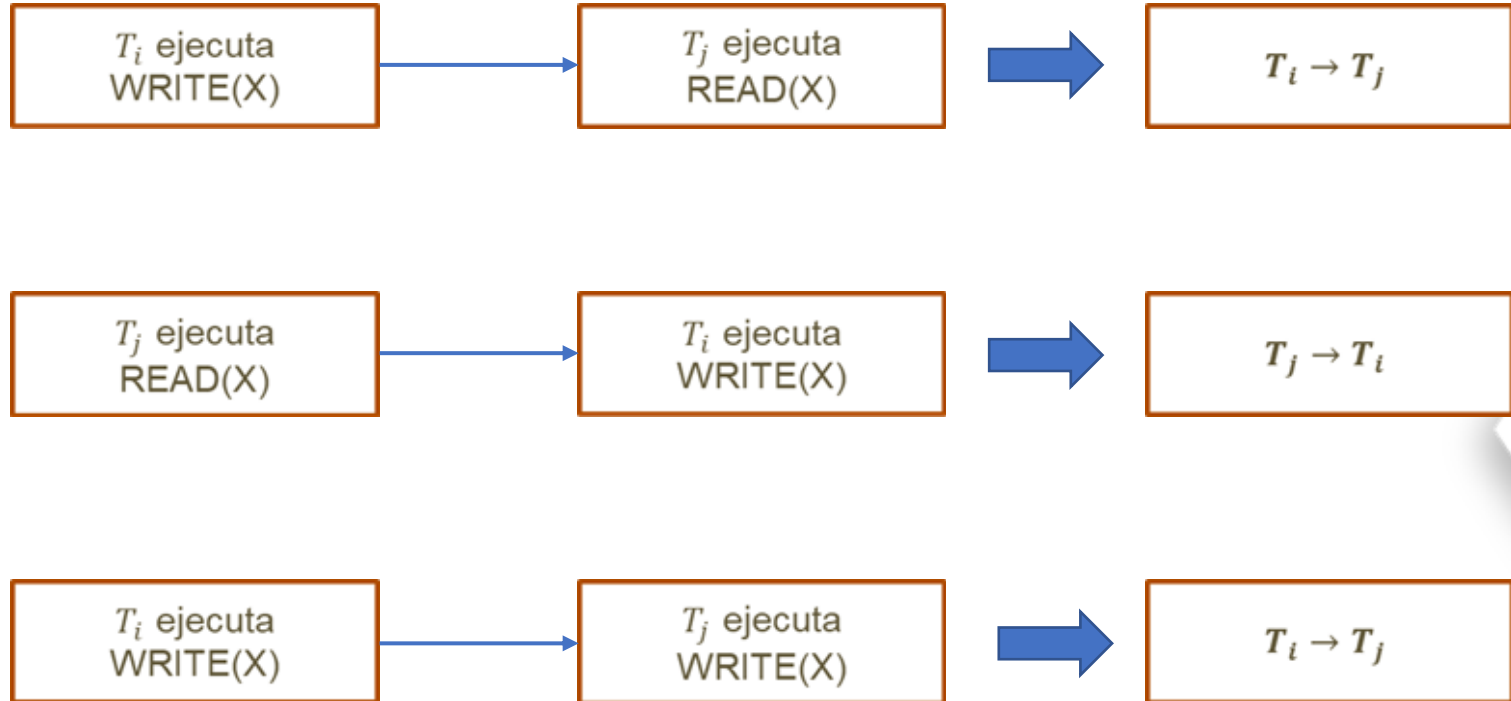
Planificación Serializable

- **Grafo de precedencia**

- Se representa como un grafo dirigido $G = (N, A)$,
con $N = \{T_1, T_2, \dots, T_n\}$, y $A = \{a_1, a_2, \dots, a_m\}$.
- Donde cada arco tiene la forma $a_i = (T_j \rightarrow T_k)$, T_j es el nodo inicial de a_i y T_k es el nodo final.
- El arco se crea si una de las operaciones de T_j aparece en el plan antes que alguna operación en conflicto de T_k .

Planificación Serializable

- Algoritmo para crear el grafo de precedencia



Planificación Serializable

- **Grafo de precedencia:** Ejemplo 1.

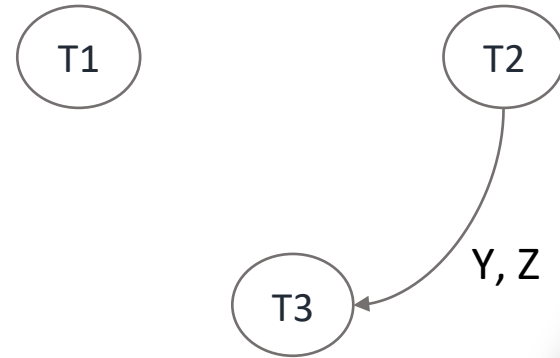
T1	T2	T3
READ(X) WRITE(X) READ(Y) WRITE(Y)	READ(Z) READ(Y) WRITE(Y) READ(X) WRITE(X)	READ(Y) READ(Z) WRITE(Y) WRITE(Z)



Planificación Serializable

- **Grafo de precedencia:** Ejemplo 1.

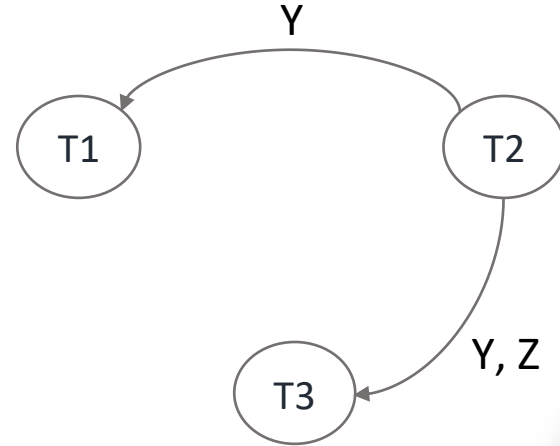
T1	T2	T3
READ(X) WRITE(X) READ(Y) WRITE(Y)	READ(Z) READ(Y) WRITE(Y) READ(X) WRITE(X)	READ(Y) READ(Z) WRITE(Y) WRITE(Z)



Planificación Serializable

- **Grafo de precedencia:** Ejemplo 1.

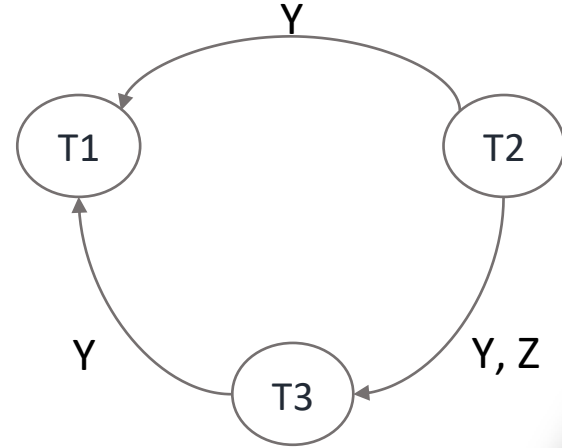
T1	T2	T3
READ(X) WRITE(X) READ(Y) WRITE(Y)	READ(Z) READ(Y) WRITE(Y) READ(X) WRITE(X)	READ(Y) READ(Z) WRITE(Y) WRITE(Z)



Planificación Serializable

- **Grafo de precedencia:** Ejemplo 1.

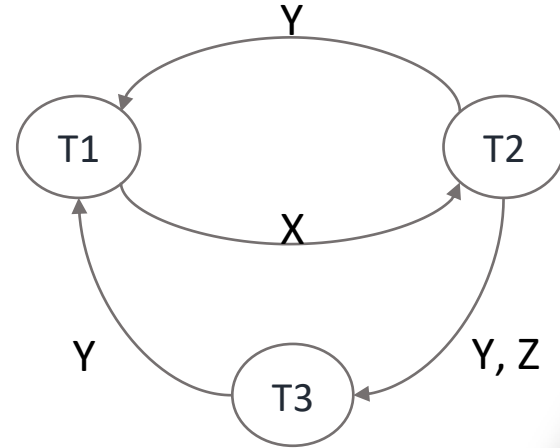
T1	T2	T3
READ(X) WRITE(X)	READ(Z) READ(Y) WRITE(Y)	READ(Y) READ(Z)
READ(Y) WRITE(Y)	READ(X) WRITE(X)	WRITE(Y) WRITE(Z)



Planificación Serializable

- **Grafo de precedencia:** Ejemplo 1.

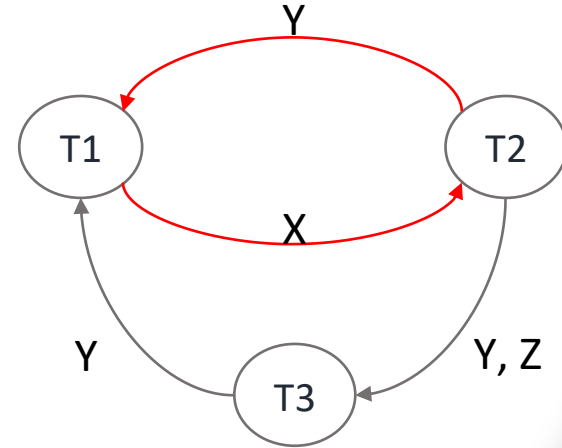
T1	T2	T3
READ(X) WRITE(X) READ(Y) WRITE(Y)	READ(Z) READ(Y) WRITE(Y) READ(X) WRITE(X)	READ(Y) READ(Z) WRITE(Y) WRITE(Z)



Planificación Serializable

- **Grafo de precedencia: Ejemplo 1.**

T1	T2	T3
READ(X) WRITE(X) READ(Y) WRITE(Y)	READ(Z) READ(Y) WRITE(Y) READ(X) WRITE(X)	READ(Y) READ(Z) WRITE(Y) WRITE(Z)



Hay un ciclo => No es serializable

Planificación Serializable

- **Grafo de precedencia:** Ejemplo 2.

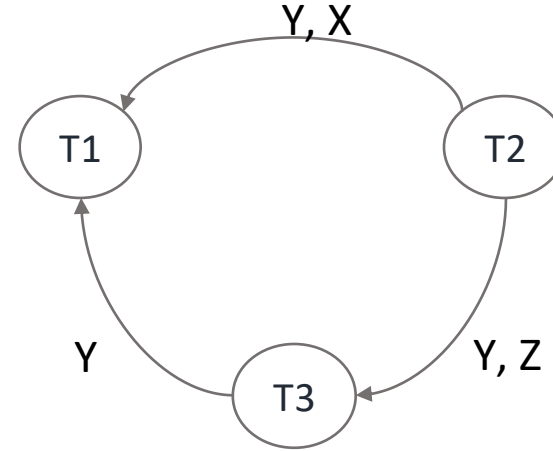
T1	T2	T3
READ(Y) WRITE(Y) READ(X) WRITE(X)	READ(Z) READ(Y) WRITE(Y) READ(X) WRITE(X)	READ(Y) READ(Z) WRITE(Y) WRITE(Z)



Planificación Serializable

- Grafo de precedencia: Ejemplo 2.

T1	T2	T3
READ(Y) WRITE(Y) READ(X) WRITE(X)	READ(Z) READ(Y) WRITE(Y) READ(X) WRITE(X)	READ(Y) READ(Z) WRITE(Y) WRITE(Z)



$T_2 \rightarrow T_1, T_2 \rightarrow T_3, T_3 \rightarrow T_1$

$T_2 \rightarrow T_3 \rightarrow T_1$

Planificación secuencial equivalente

Planificación Serializable

- Grafo de precedencia: Ejemplo 2.

T1	T2	T3
READ(Y) WRITE(Y) READ(X) WRITE(X)	READ(Z) READ(Y) WRITE(Y) READ(X) WRITE(X)	READ(Y) READ(Z) WRITE(Y) WRITE(Z)

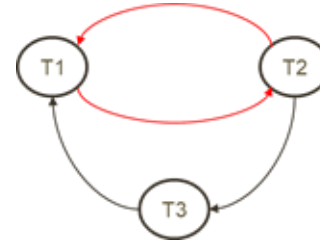
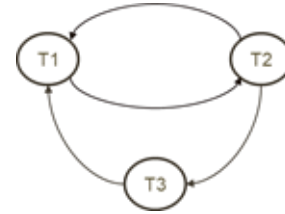
T1	T2	T3
READ(Y) WRITE(Y) READ(X) WRITE(X)	READ(Z) READ(Y) WRITE(Y) READ(X) WRITE(X)	READ(Y) READ(Z) WRITE(Y) WRITE(Z)

$$T_2 \rightarrow T_3 \rightarrow T_1$$

Planificación secuencial equivalente

Planificación Serializable: Pasos Generales

1. Crear la estructura de grafo a partir de la planificación concurrente de transacciones.
1. Algoritmo para verificar si se arma al menos un ciclo dentro del grafo.
1. Caso que no haya ciclos, algoritmo debe indicar al menos una planificación secuencial equivalente.



$T_2 \rightarrow T_1, T_2 \rightarrow T_3, T_3 \rightarrow T_1$

$T_2 \rightarrow T_3 \rightarrow T_1$

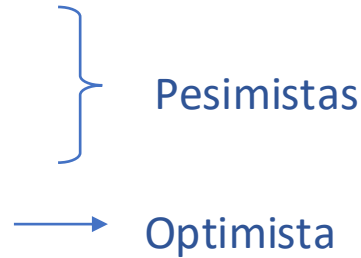
¿Cómo garantizamos una planificación serializable?

Técnicas de control de concurrencia

Técnicas de control de concurrencia

- El objetivo es garantizar que la ejecución de un conjunto de transacciones sea serializable:

- Bloqueos
- Marcas de tiempo
- Validación

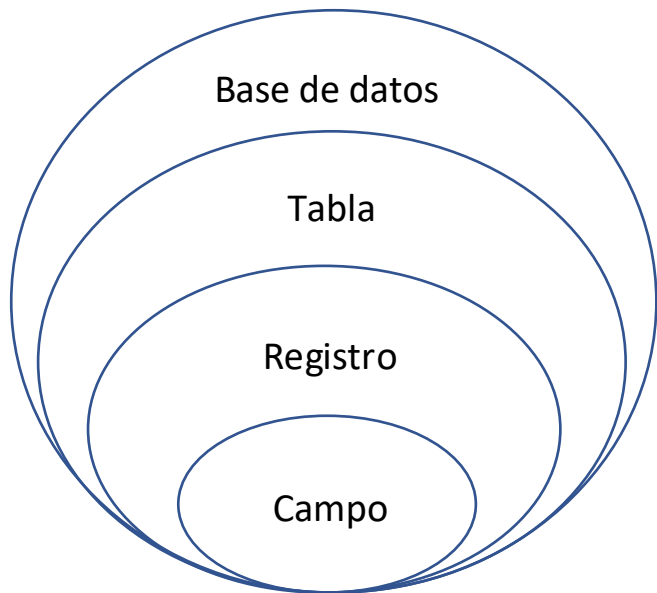


- **Pesimistas:** Garantizan la seriabilidad antes de la ejecución.

- **Optimistas:** Se ejecutan y luego se verifica si la ejecución es serializable.

Técnicas de Bloqueo

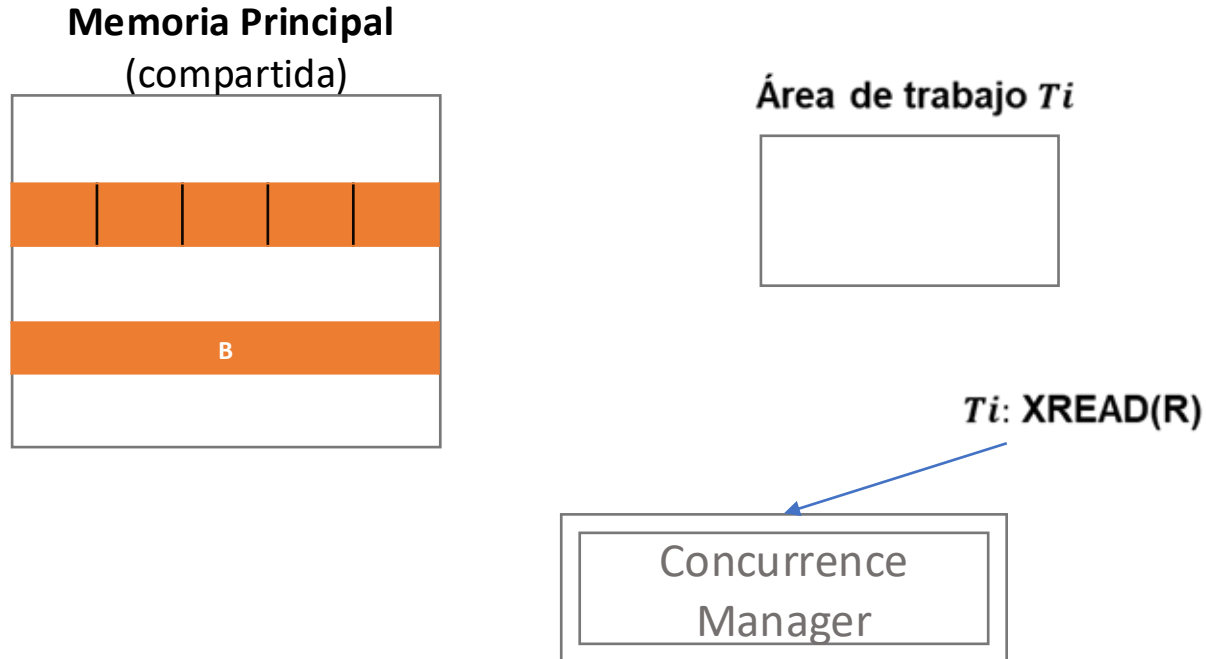
- El **bloqueo** regula el acceso concurrente a objetos compartidos (buffer de datos).
 - **Granularidad:** pueden aplicarse a diferentes unidades de dato.



- Grano más fino
- Mayor concurrencia
- Mayores posibilidades de interbloqueo
- Mas costos de manejo de concurrencia

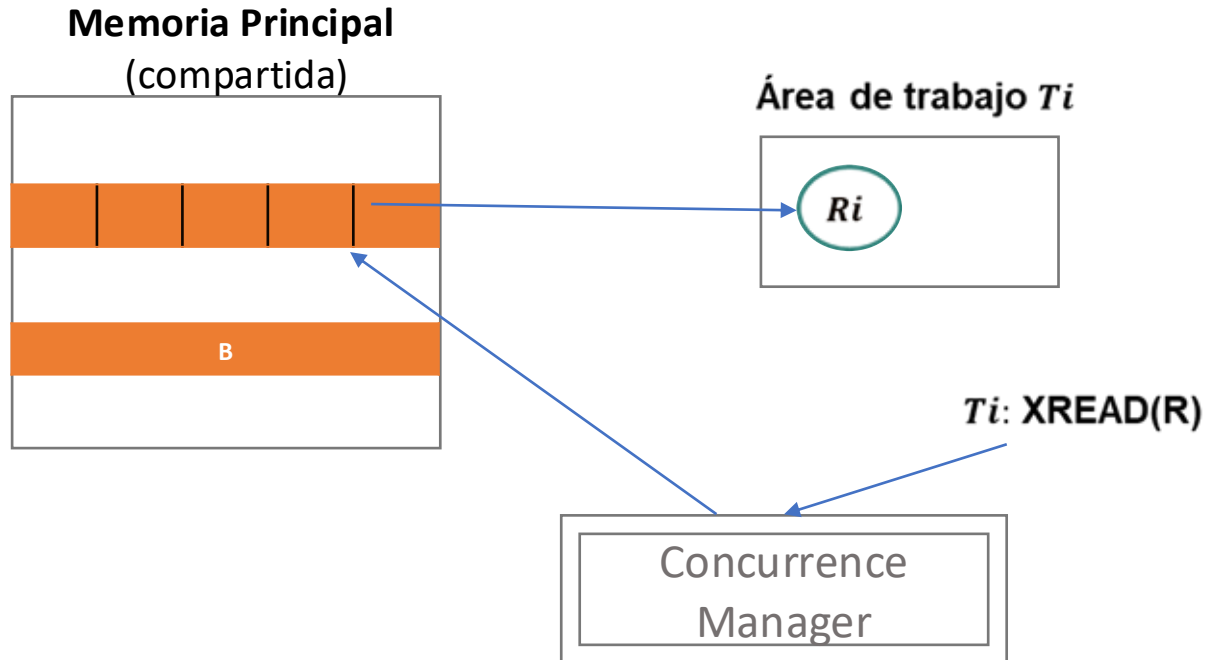
Técnicas de Bloqueo

- Bloqueo Exclusivo (Protocolo PX)



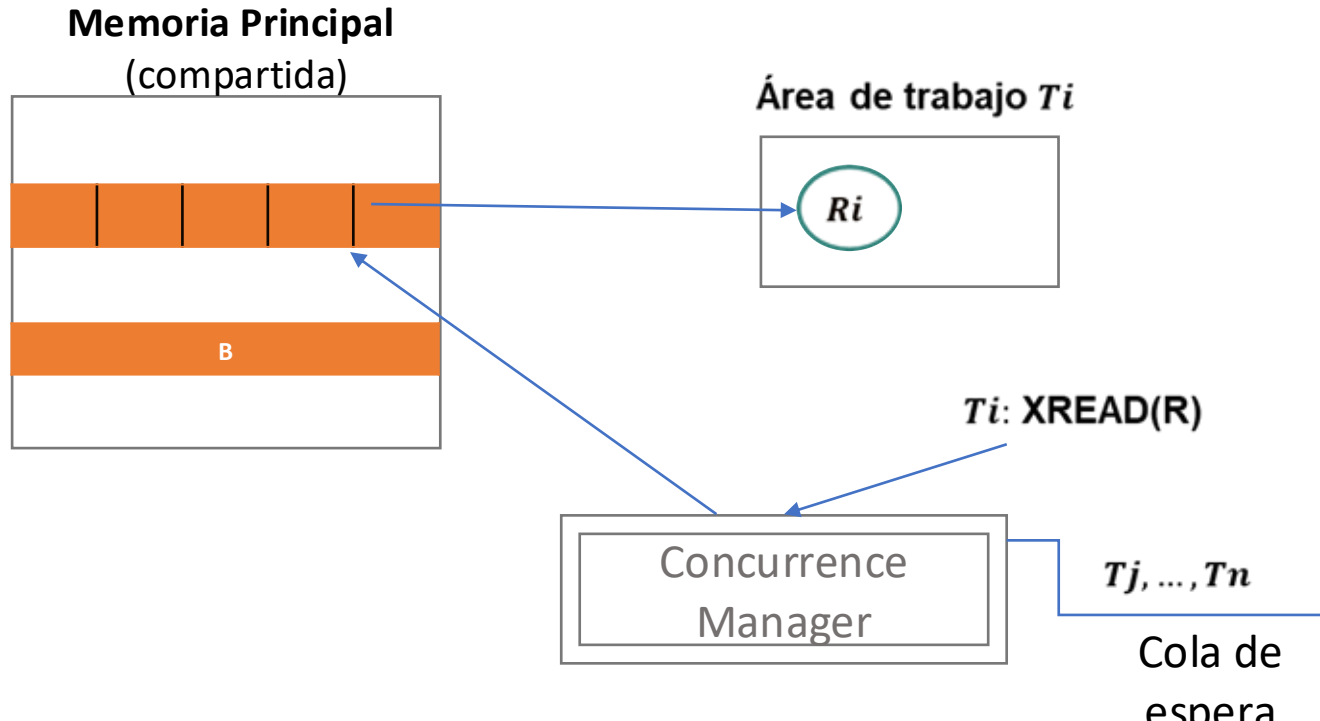
Técnicas de Bloqueo

- Bloqueo Exclusivo (Protocolo PX)



Técnicas de Bloqueo

- Bloqueo Exclusivo (Protocolo PX)



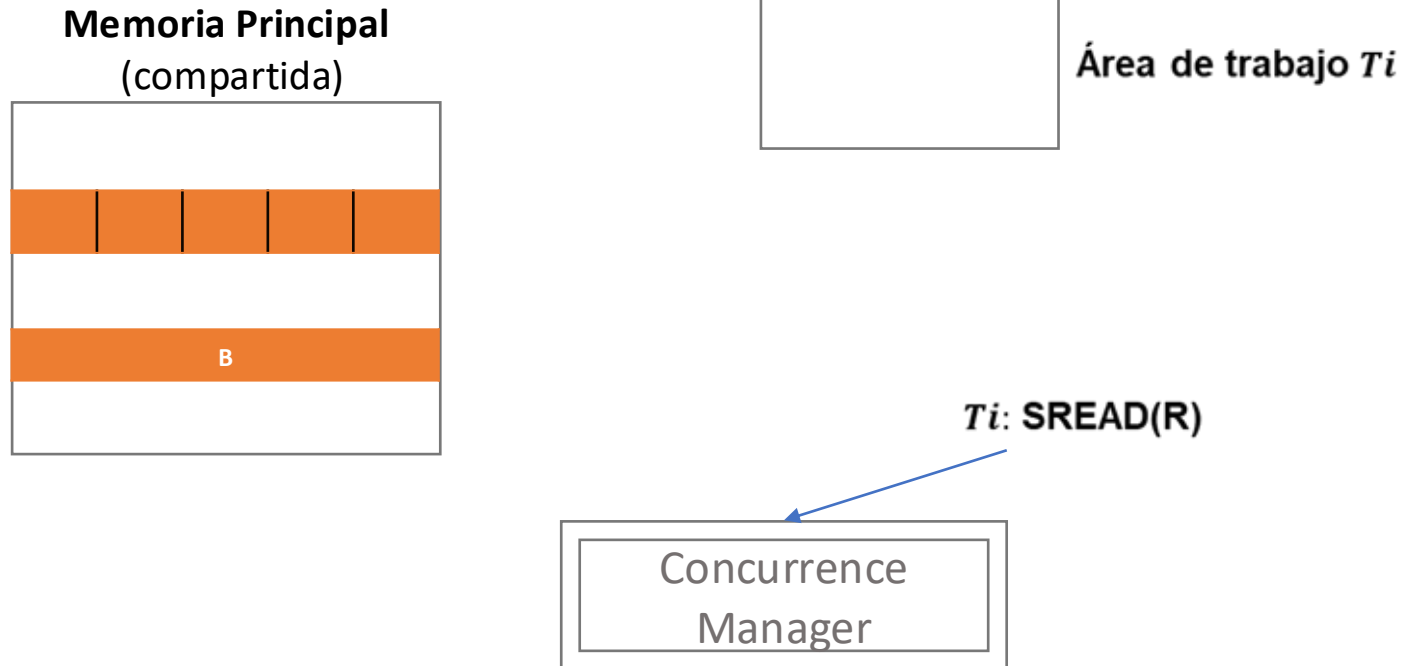
Técnicas de Bloqueo

- Bloqueo Exclusivo (Protocolo PX)

T1	Tiempo	T2
XREAD(R) COPY R.F INTO ATEMP	t1	
	t2	XREAD(R)
XWRITE(R): REPLACE R.F BY ATEMP + 1	t3	<i>wait</i>
XRELEASE(R)	t4	<i>wait</i>
	t5	<i>(reinicia XREAD(R))</i> COPY R.F INTO BTEMP
	t6	XWRITE(R): REPLACE R.F BY BTEMP*2
	t7	XRELEASE(R)

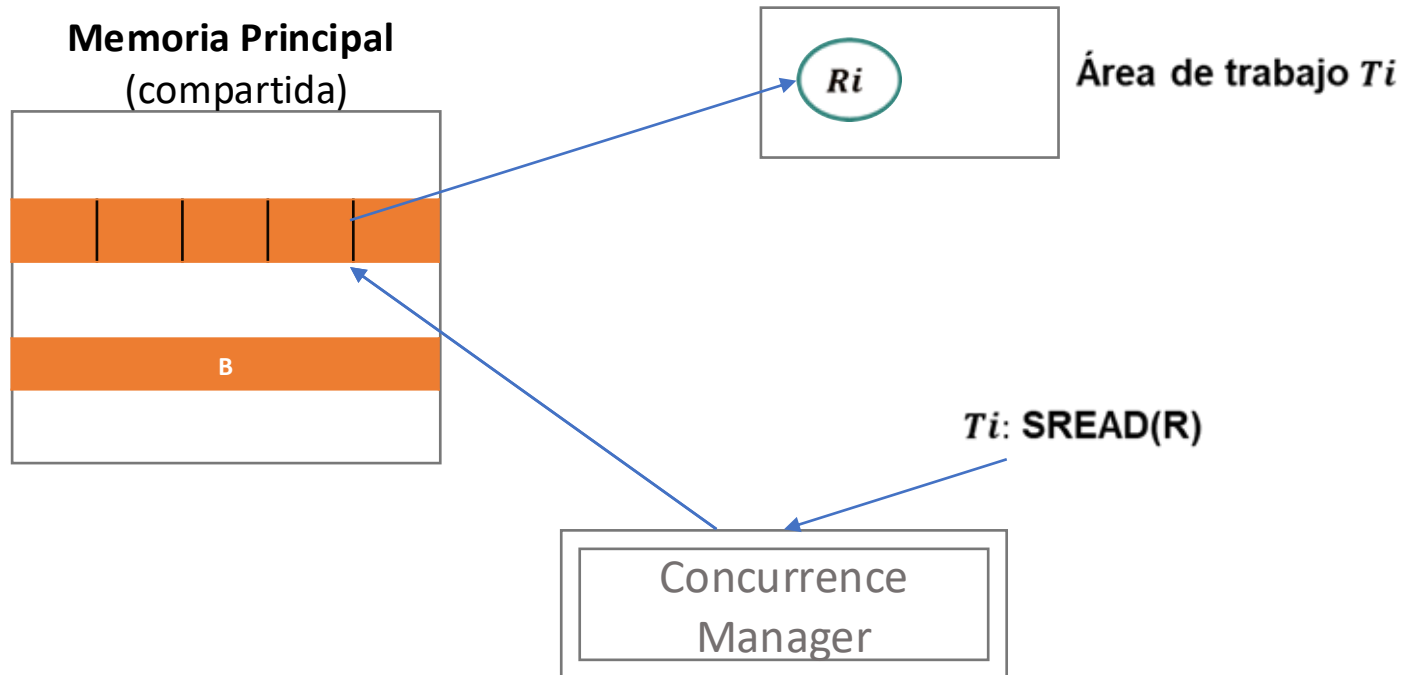
Técnicas de Bloqueo

- Bloqueo Compartido (Protocolo PS)



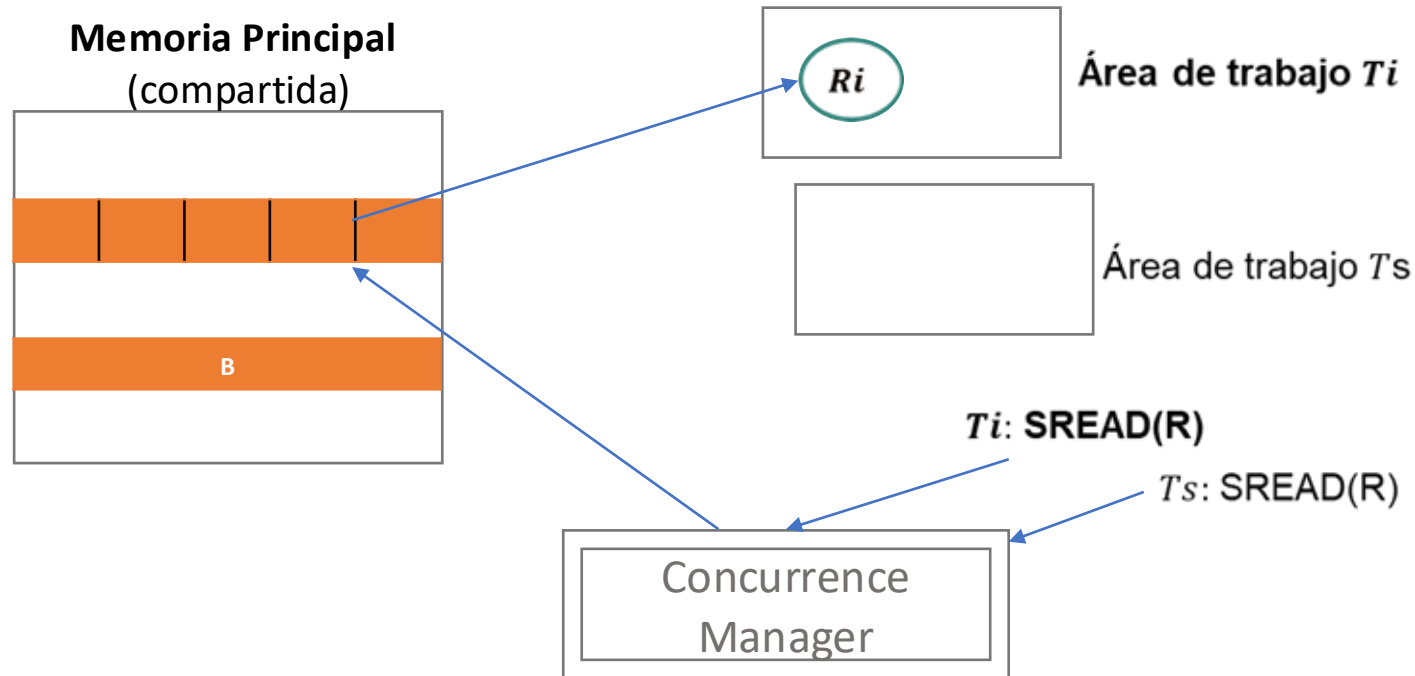
Técnicas de Bloqueo

- Bloqueo Compartido (Protocolo PS)



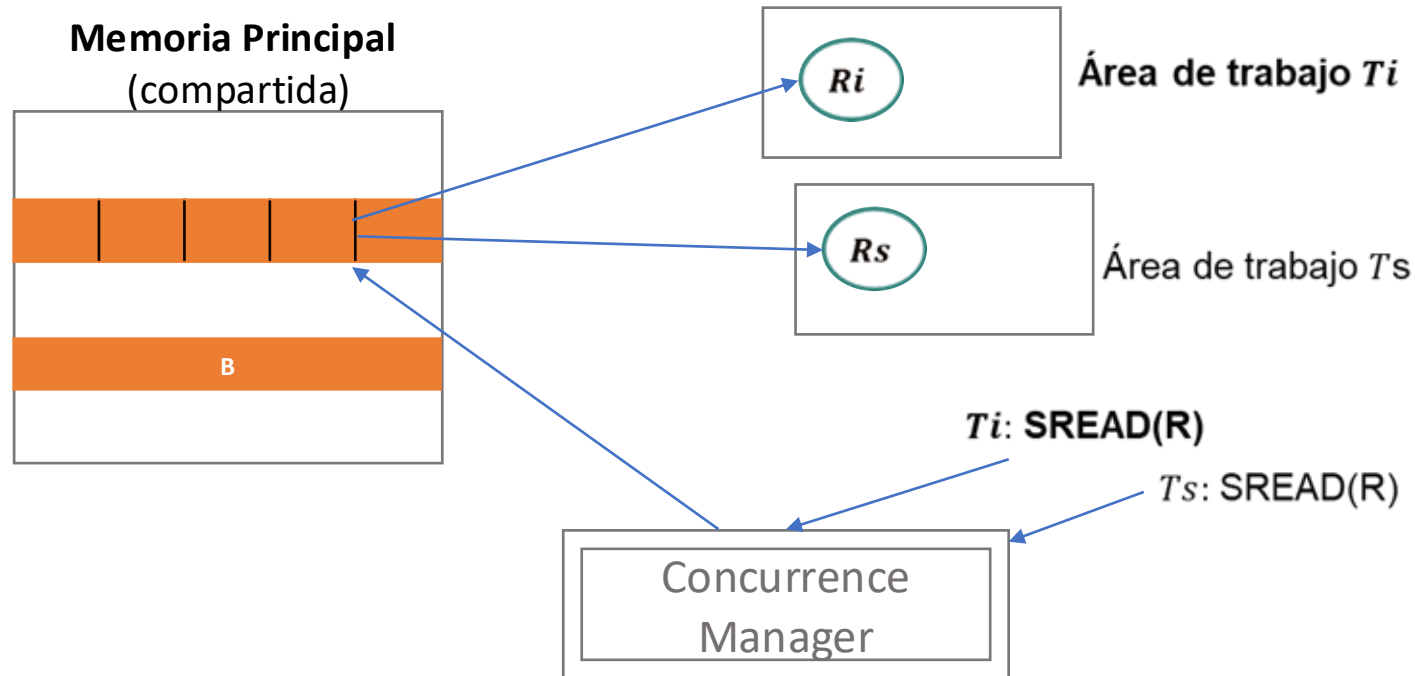
Técnicas de Bloqueo

- Bloqueo Compartido (Protocolo PS)



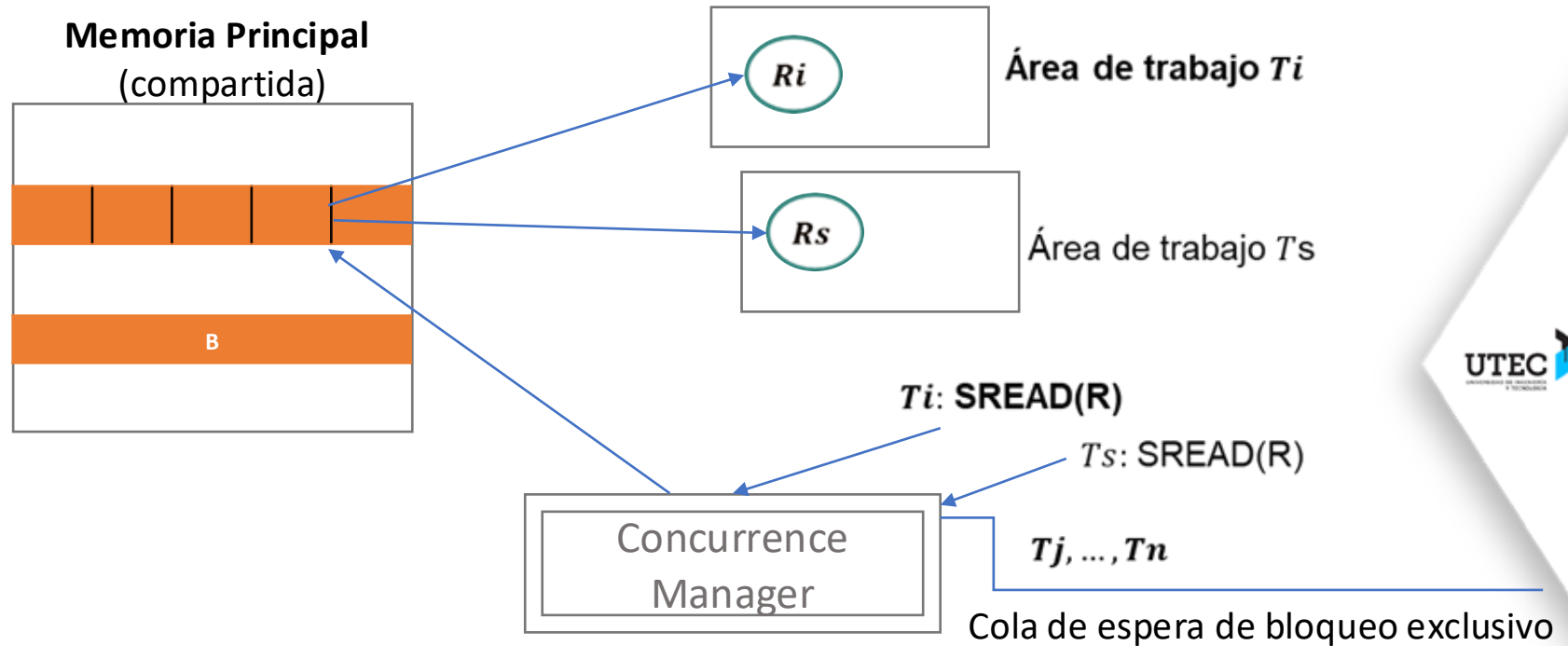
Técnicas de Bloqueo

- Bloqueo Compartido (Protocolo PS)



Técnicas de Bloqueo

- Bloqueo Compartido (Protocolo PS)



Técnicas de Bloqueo

● Bloqueo Compartido (Protocolo PS)

ACC1=40

ACC2=50

ACC3=30

T1	Tiempo	T2
SREAD(ACC1) SUM = SUM + ACC1 (40)	t1	
SREAD(ACC2) SUM = SUM + ACC2 (90)	t2	
	t3	XREAD(ACC3) ACC3 = ACC3 - 10 XWRITE(ACC3) :
SREAD(ACC3) <i>wait</i>	t4	
<i>wait</i>	t5	XRELEASE(ACC3) SREAD(ACC1)
SUM = SUM + ACC3 (110)	t6	
	t7	COMMIT

Técnicas de Bloqueo

- Bloqueo Compartido (Protocolo PS)

T1	Tiempo	T2
SREAD(ACC1) SUM = SUM + ACC1	t1	
SREAD(ACC2) SUM = SUM + ACC2	t2	
	t3	XREAD(ACC3) ACC3 = ACC3 - 10 XWRITE(ACC3)
	t4	SREAD(ACC1)
	t5	XWRITE(ACC1)
SREAD(ACC3)	t6	
	t7	

¿Qué pasa aquí?

Técnicas de Bloqueo

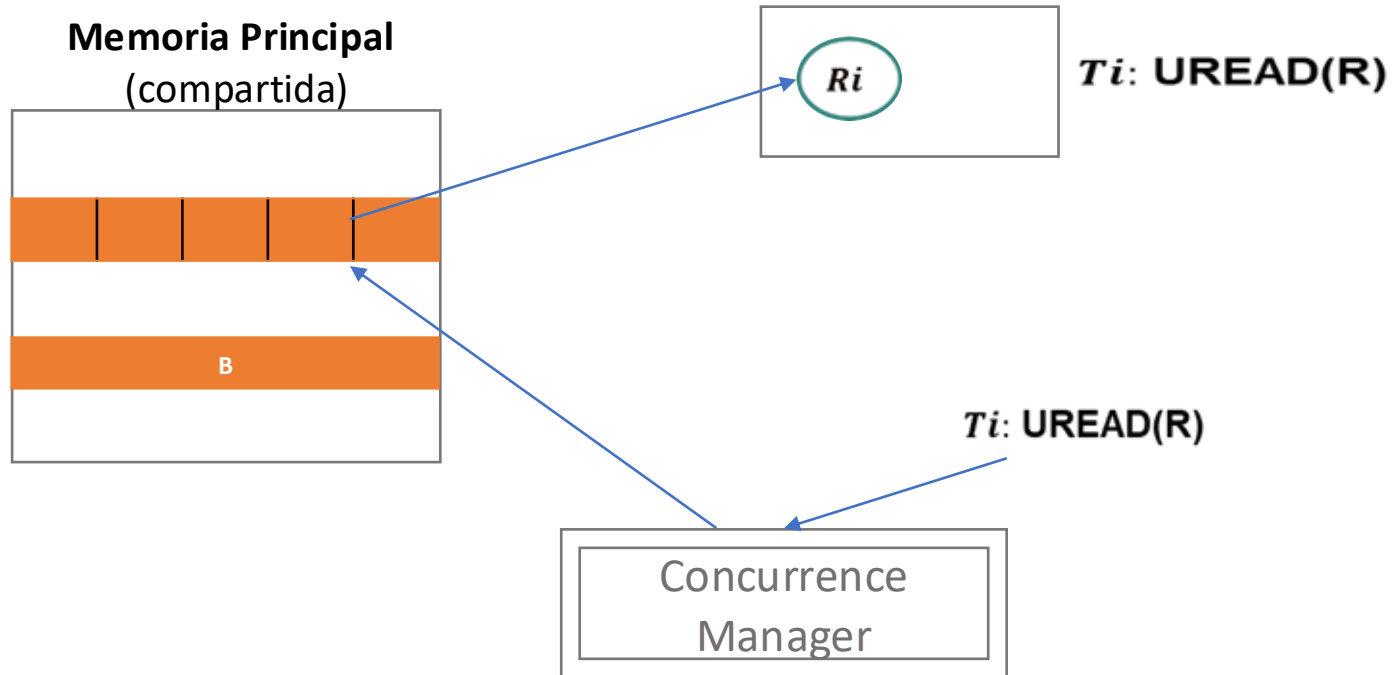
- Bloqueo Compartido (Protocolo PS)

¿Qué pasa aquí?

T1	Tiempo	T2
SREAD(ACC1) SUM = SUM + ACC1	t1	
SREAD(ACC2) SUM = SUM + ACC2	t2	
	t3	XREAD(ACC3) ACC3 = ACC3 - 10 XWRITE(ACC3)
	t4	SREAD(ACC1)
	t5	XWRITE(ACC1) <i>wait</i>
SREAD(ACC3) <i>wait</i>	t6	<i>wait</i>
<i>wait</i>	t7	<i>wait</i>

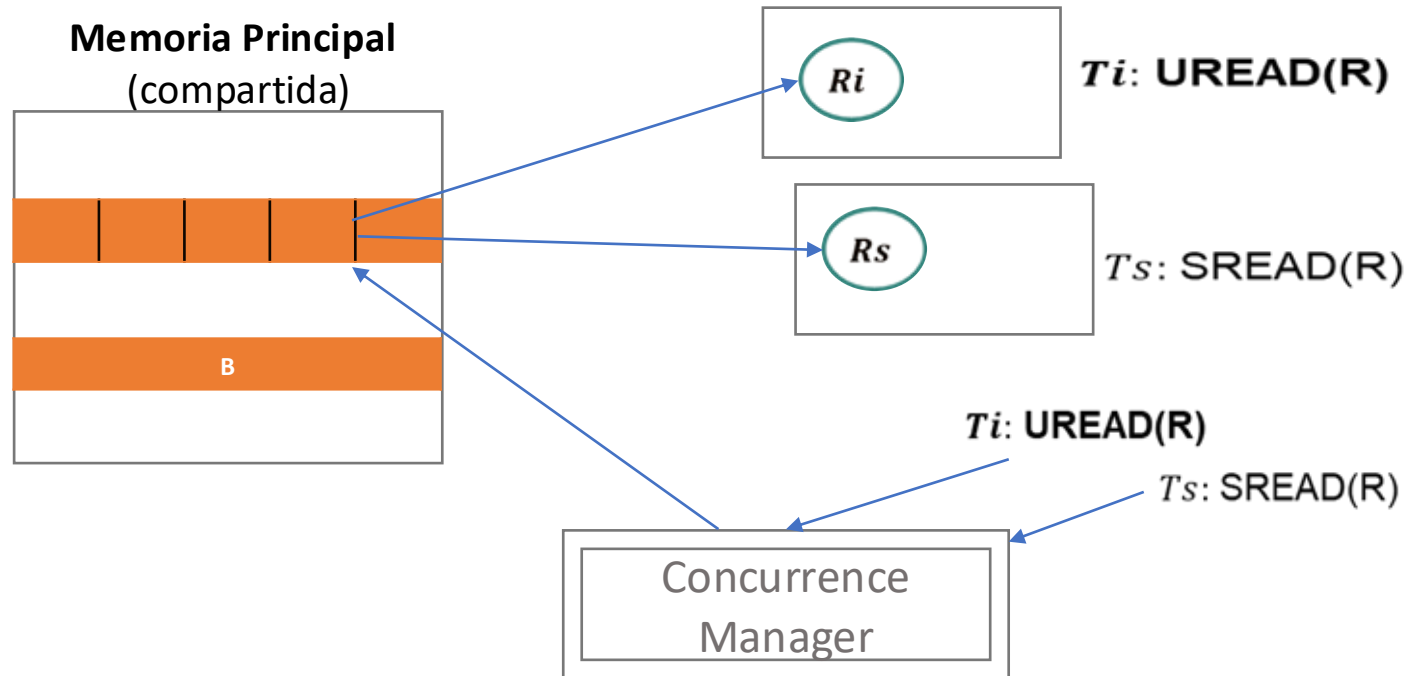
Técnicas de Bloqueo

- Bloqueo Actualización (Protocolo PU)



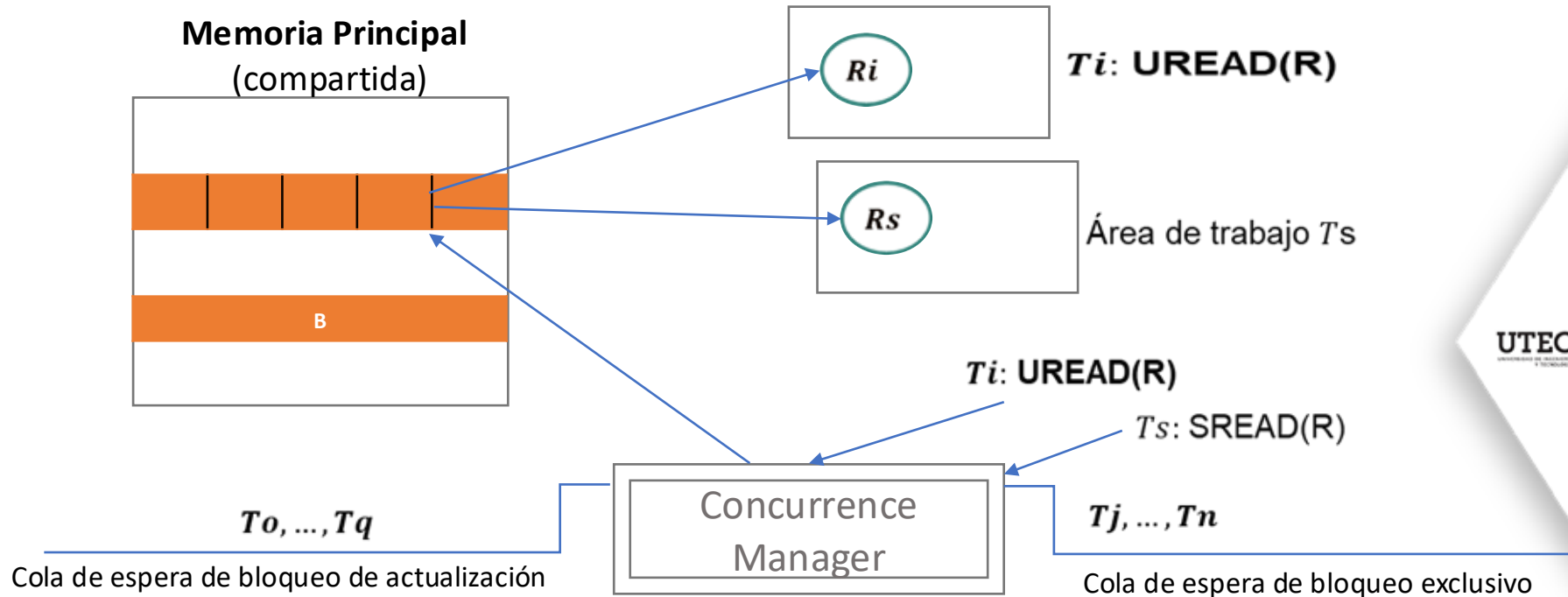
Técnicas de Bloqueo

- Bloqueo Actualización (Protocolo PU)



Técnicas de Bloqueo

- Bloqueo Actualización (Protocolo PU)



Técnicas de Bloqueo

- Bloqueo Actualización (Protocolo PU)

T1	Tiempo	T2
SREAD(R) COPY R.F INTO ATEMP	t1	
	t2	SREAD(R) COPY R.F INTO BTEMP
XWRITE(R) <i>wait</i>	t3	
<i>wait</i>	t4	XWRITE(R) <i>wait</i>
<i>wait</i>	t5	<i>wait</i>

Interbloqueo
(deadlock)

Técnicas de Bloqueo

- Bloqueo Actualización (Protocolo PU)

T1	Tiempo	T2
UREAD(R) COPY R.F INTO ATEMP	t1	
	t2	UREAD(R) <i>wait</i>
UWRITE(R) REPLACE R.F BY ATEMP + 1	t3	<i>wait</i>
COMMIT	t4	<i>reinicia</i> COPY R.F INTO BTEMP
	t5	UWRITE(R) REPLACE R.F BY BTEMP + 1
	t5	COMMIT

Técnicas de Bloqueo

- Algoritmo para bloqueo exclusivo:

Data: An element X of the database

Result: Update of variable $BLOCK(X)$

B: **if** $BLOCK(X) = 0$ **then**

| $BLOCK(X) \leftarrow 2$;

else

| wait(until $BLOCK(X) = 0$ and DBMS wakes up the
| transaction);

| go to B;

end

Técnicas de Bloqueo

- Algoritmo para bloqueo compartido:

```
Data: An element  $X$  of the database  
Result: Update of variable  $BLOCK(X)$   
B: if  $BLOCK(X) = 0$  then  
|    $BLOCK(X) \leftarrow 1$ ;  
|   num_read  $\leftarrow 1$ ;  
else  
|   if  $BLOCK(X) = 1$  then  
|   |   num_read  $\leftarrow$  num_read + 1;  
|   else  
|   |   wait(until  $BLOCK(X) = 0$  and DBMS wakes up the  
|   |   |   transaction);  
|   |   go to B;  
|   end  
end
```

¿Cómo modificamos este algoritmo para incluir el caso de bloqueo con protocolo PU?

Técnicas de Bloqueo

- Algoritmo de desbloqueo:

```
Data: An element X of the database  
Result: Update of variable BLOCK(X)  
B: if BLOCK(X) = 2 then  
    |   BLOCK(X) ← 0;  
    |   Wake up the waiting transactions;  
else  
    |   if BLOCK(X) = 1 then  
    |       |   num_read = num_read - 1;  
    |       |   if num_read = 0 then  
    |       |       |   BLOCK(X) ← 0;  
    |       |       |   Wake up the waiting transactions;  
    |       |       end  
    |       end  
    end  
end
```

¿Cómo modificamos este algoritmo para incluir el caso de desbloqueo con protocolo PU?

Técnicas de Bloqueo

El problema de las técnicas de bloqueo es que puede producirse un interbloqueo o bloqueo mortal.

Esto es cuando dos o más transacciones están esperando cada una de ellas que la otra libere algún objeto antes de seguir.

2.

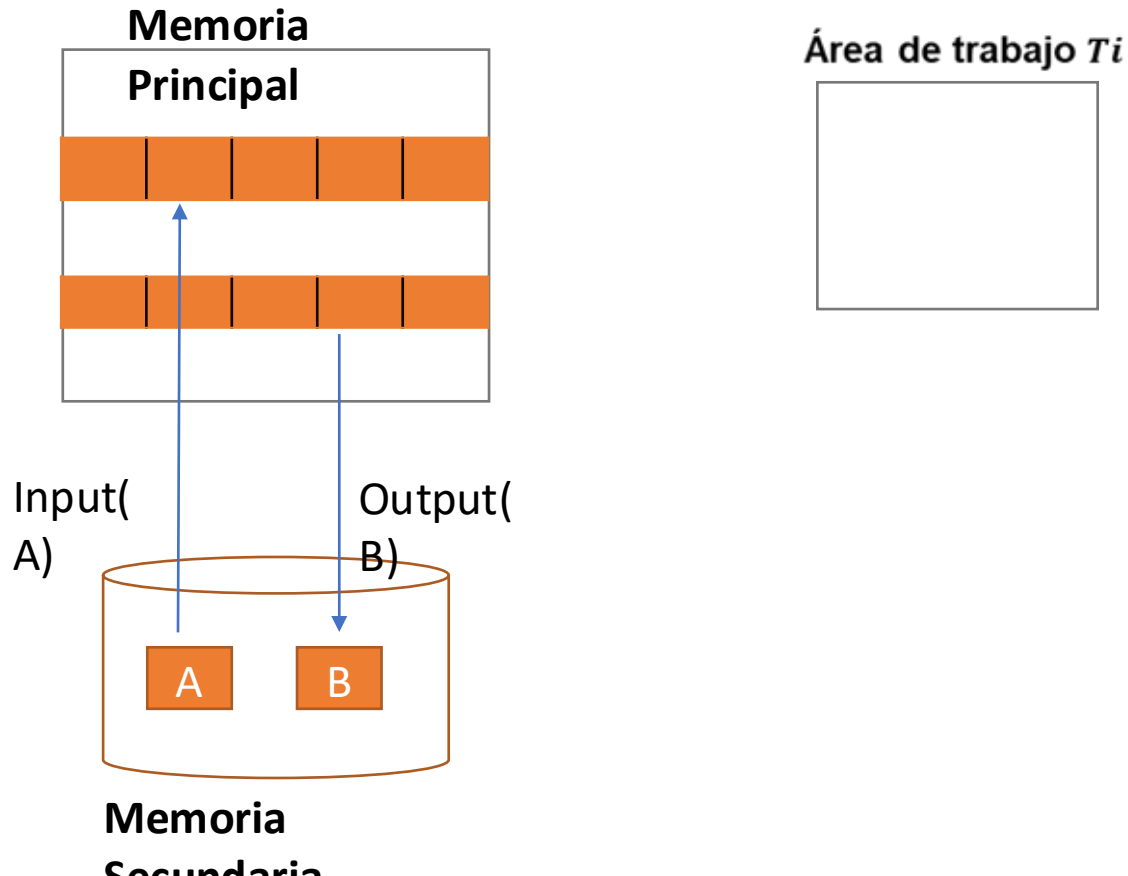


Database Recovery

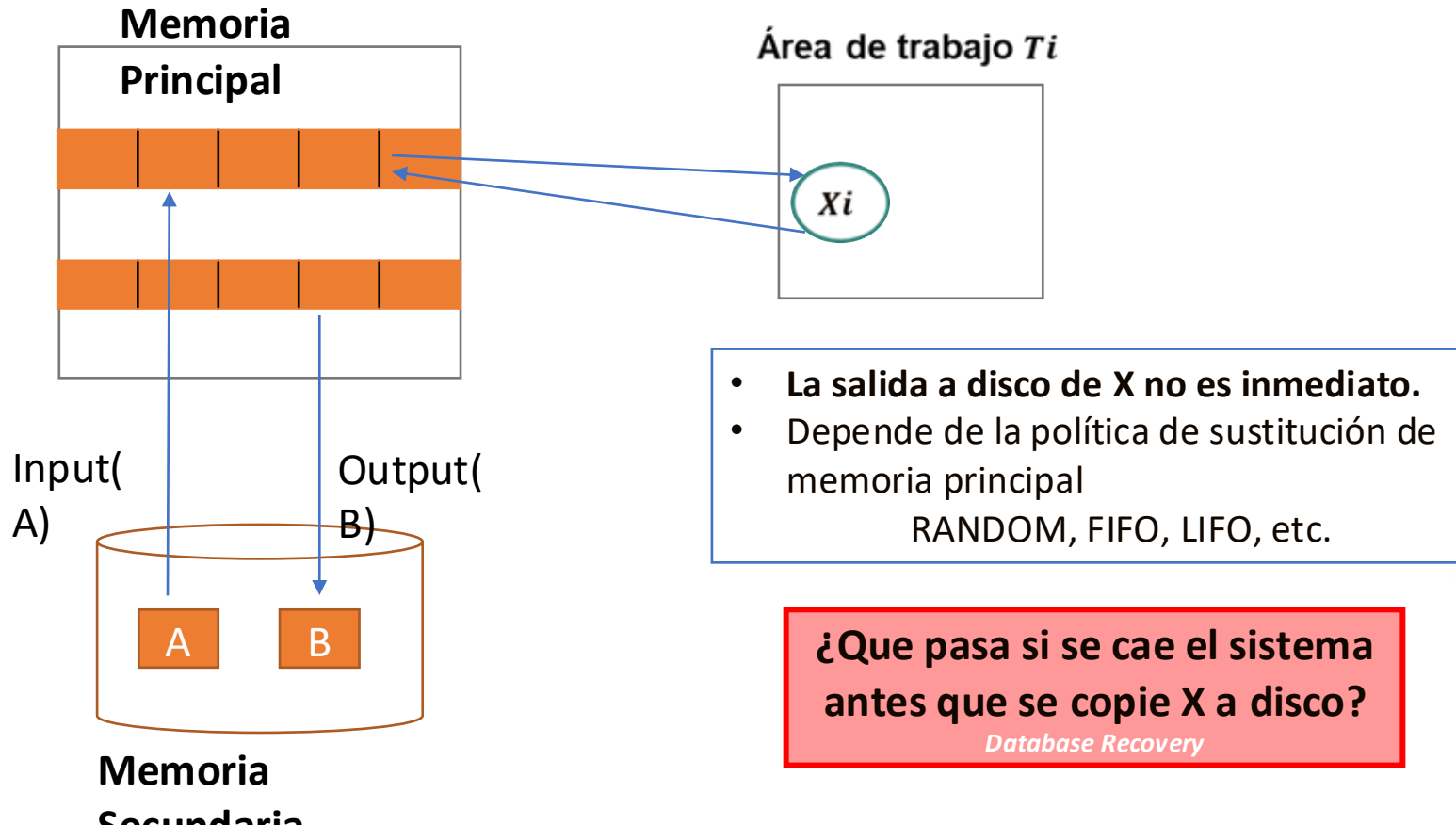
WRITE-AHEAD LOG (WAL) y Recovery

*Database Management Systems 3rd Edition,
Ramakrishnan, Chapter 18*

Acceso a datos por una transacción



Acceso a datos por una transacción



DB Recovery

- En un sistema gestor de base de datos, restaurar la BD significa:
 - Re-almacenar la base de datos en un **estado correcto**, lo más reciente posible, si la falla ha hecho que la BD quede en un **estado incorrecto**.
 - Implica **redundancia** (almacenar los estados correctos) de manera que la BD pueda ser reconstruida con estos.

DB Recovery

- **Tipo de fallas en un SGBD:**

Fallas locales a la transacción

- a) Son detectados por el código de la aplicación y son manejados por este. Ej. Condición de fondos insuficiente en la transferencia.
- b) No son explícitamente manejadas por el código de aplicación. Ej. Overflow aritmético.

Fallas en el sistema

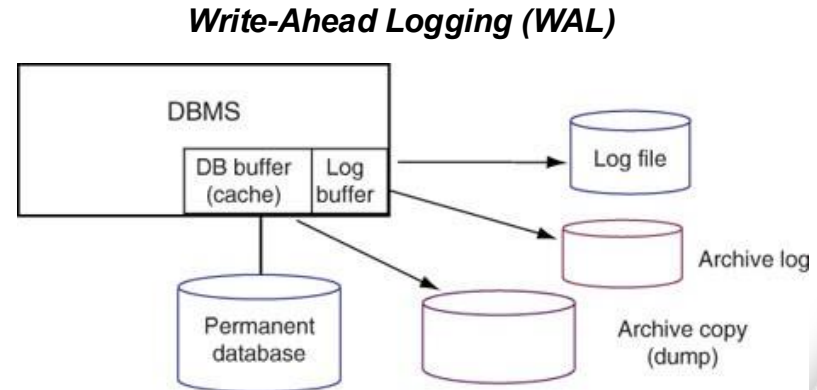
- Afectan a todas las transacciones que se están realizando en ese momento, pero no daña la BD (física). Ej. Falla del CPU, falla eléctrico, falla del software.

Fallas en los medio de almacenamiento

- Dañan en la BD o una porción de esta y afectan a todas las transacciones que estaban usando esa porción en el momento de la falla. Ej. Cabeza lectora que raya el disco.

Recuperación basado en Log

- Registro de Log:
 - Es una secuencia de eventos que documentan cada operación crítica (INSERT, UPDATE, DELETE).
- Estrategias de modificación
 - Modificación diferida
 - Modificación inmediata
- Proceso de recuperación con Log
 - Tres pasadas y Dos pasadas
 - Basado en Checkpoints



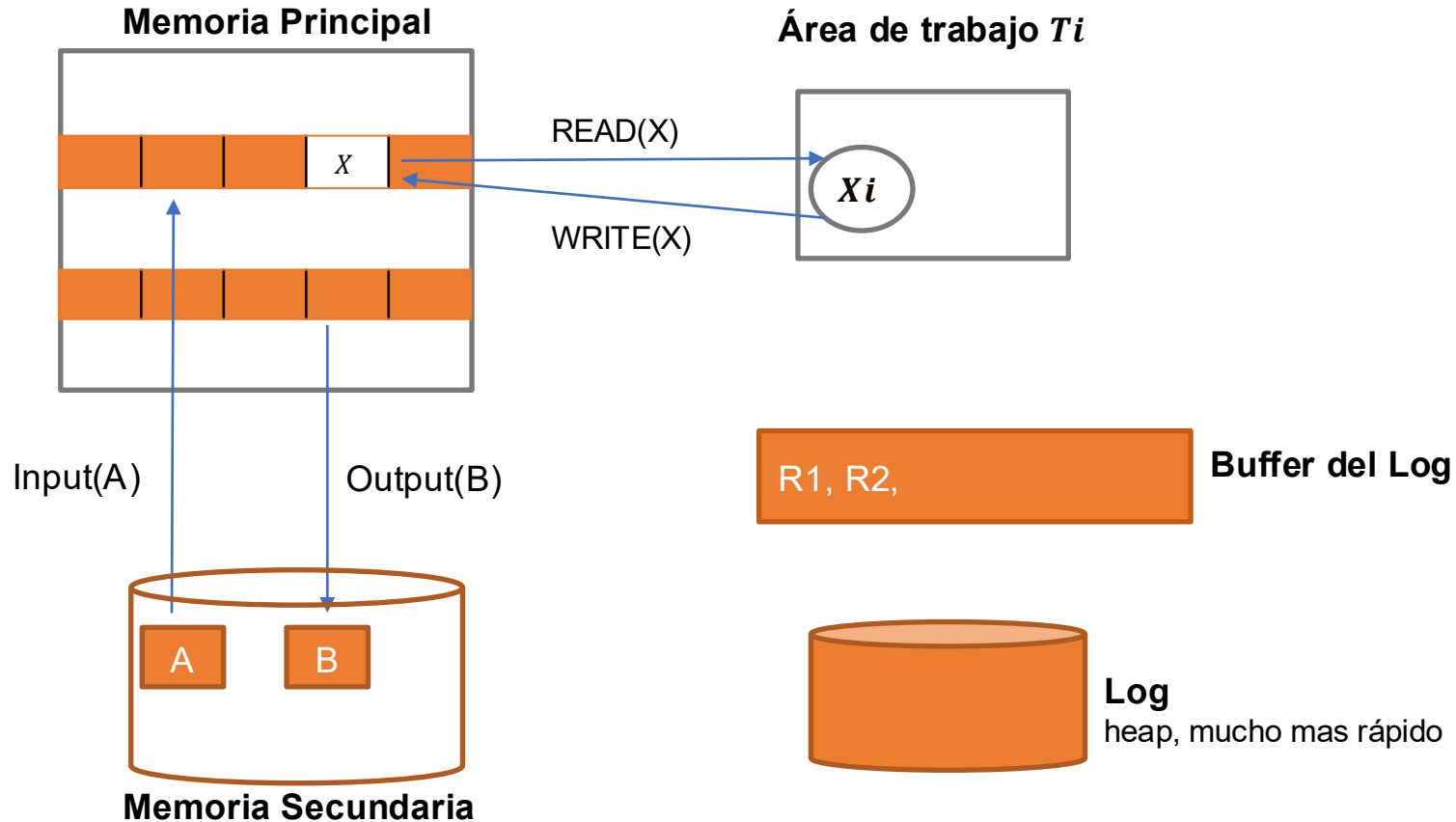
Log = Registro histórico

Recuperación basado en Log

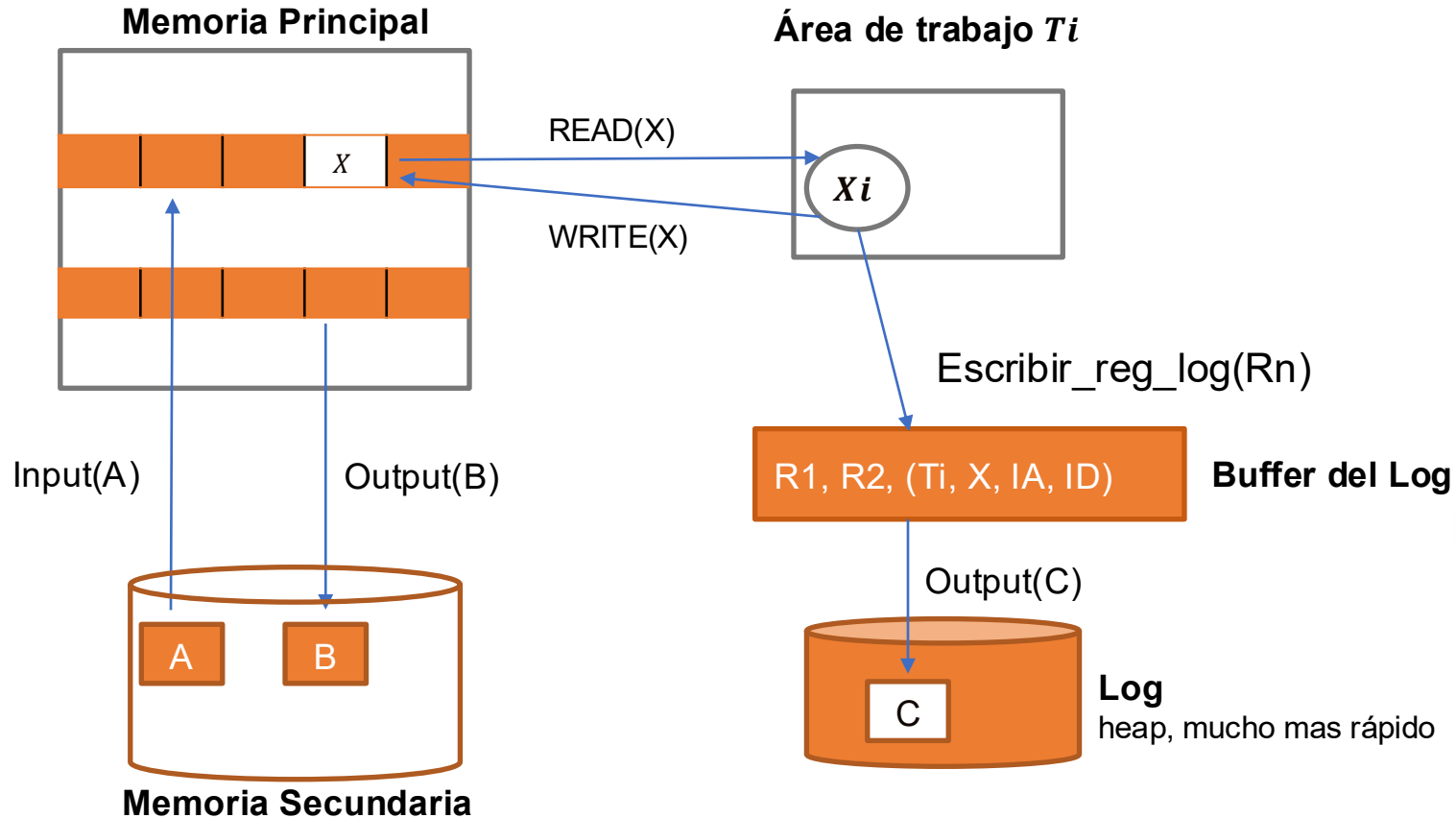
- Estructura de los registros de Log
 - Inicio de la transacción T_i : $\langle T_i, BT \rangle$
 - Fin de la transacción T_i : $\langle T_i, ET \rangle$ o $\langle T_i, Commit \rangle$ *culminación exitosa* o $\langle T_i, Abort \rangle$ *culminación fallida*.
 - Cambios en los registros de la BD: $\langle T_i, X, IA, ID \rangle$

- T_i : Identificador de la transacción
- IA : Imagen o estado antes de la modificación.
- ID : imagen o estado después de la modificación.

Recuperación basado en Log



Recuperación basado en Log



Recuperación basado en Log

Deshacer transacción:

- Acción de colocar las imágenes antes (estado anterior) de los registros modificados por una transacción.

Rehacer transacción

- Acción de escribir la imagen después (estado posterior) de los registros modificados por una transacción.

Estrategias de modificación

- **Modificación inmediata:**

- Permite las operaciones WRITE de una transacción mientras ésta está activa (modificación no comprometidas)
- Si el sistema falla se debe usar los valores del Log para restaurar los elementos de datos de los valores anteriores a la transacción.
- En el Log
 - Los registros de actualización son de la forma $\langle T_i, X, IA, ID \rangle$
 - Se realizan los cambios en el Log antes de la actualización real a la BD.

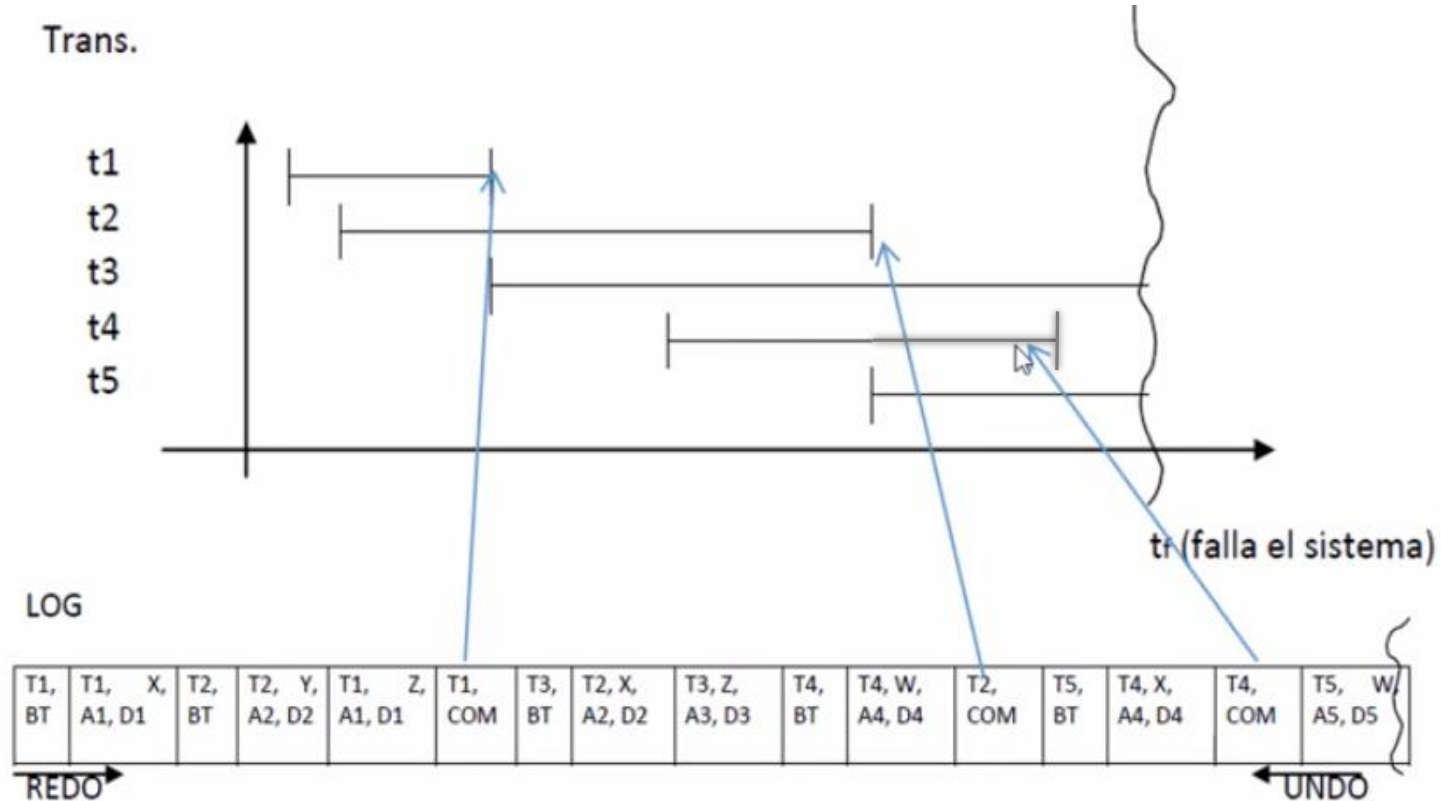
Estrategias de modificación

- **Modificación diferida:**

- Retarda las operaciones WRITE de una transacción hasta que se compromete parcialmente.
- Si el sistema falla antes de que la transacción se complete, se ignora en el proceso de recuperación.
- En el Log no se registra las imágenes antes, por lo que:
 - Los registros de actualización son de la forma $\langle T_i, X, ID \rangle$
 - No se deshace.

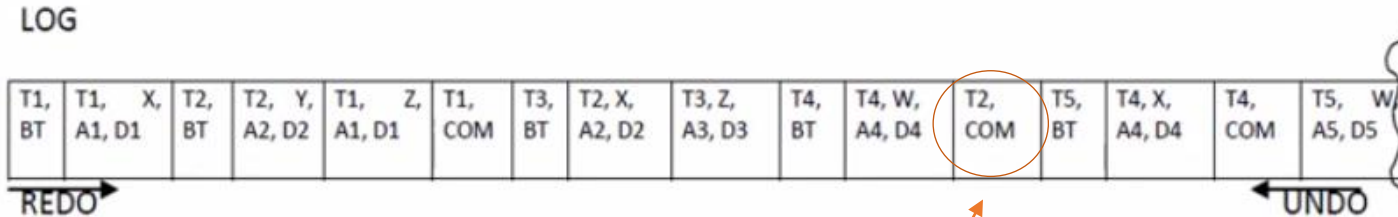
Proceso de recuperación

- Tres pasadas:



Proceso de recuperación

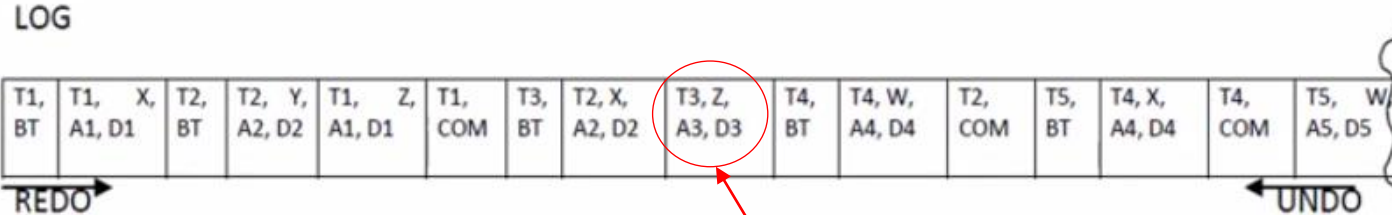
- Tres pasadas:



- Se recorre el log del fin al inicio para crear dos listas
 - REDO para transacciones terminadas

Proceso de recuperación

- Tres pasadas:



- Se recorre el log del fin al inicio para crear dos listas
 - REDO para transacciones terminadas
 - UNDO para transacciones no terminadas

Proceso de recuperación

- Tres pasadas:

LOG

T1, BT	T1, X, A1, D1	T2, BT	T2, Y, A2, D2	T1, Z, A1, D1	T1, COM	T3, BT	T2, X, A2, D2	T3, Z, A3, D3	T4, BT	T4, W, A4, D4	T2, COM	T5, BT	T4, X, A4, D4	T4, COM	T5, W/ A5, D5
REDO →										← UNDO					

- Se recorre el log del fin al inicio para crear dos listas
 - REDO para transacciones terminadas
 - UNDO para transacciones no terminadas
- Se recorre el log de inicio a fin para Rehacer las transacciones en REDO
- Se recorre el log de fin a inicio para Deshacer las transacciones en UNDO.

Proceso de recuperación

- Tres pasadas:

LOG

T1, BT	T1, X, A1, D1	T2, BT	T2, Y, A2, D2	T1, Z, A1, D1	T1, COM	T3, BT	T2, X, A2, D2	T3, Z, A3, D3	T4, BT	T4, W, A4, D4	T2, COM	T5, BT	T4, X, A4, D4	T4, COM	T5, W, A5, D5
REDO →										← UNDO					

- Se recorre el log del fin al inicio para crear dos listas

- REDO para transacciones terminadas

- UNDO para transacciones no terminadas

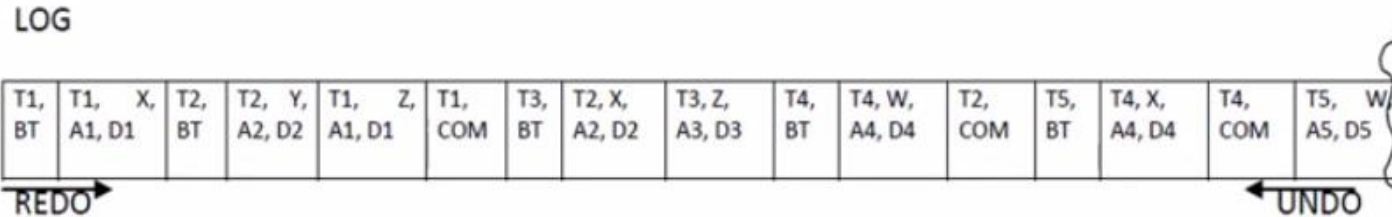
- Se recorren las listas para:

- Se recorren las listas para:

En modificación diferida las transacciones en UNDO solo se procede a anularlas, ya que ninguna de sus operaciones de escritura afectaron a la base de datos

Proceso de recuperación

- **Dos pasadas**



- Se recorre el log del fin al inicio para :
 - Crear lista REDO
 - Deshacer las transacciones no terminadas
- Se recorre el log de inicio a fin para Rehacer las transacciones en REDO

El Log puede llegar a tener muchos registros para Rehacer.
Muchas de estas transacciones para rehacer ya están en la BD

Proceso de recuperación

- **Problemas:**

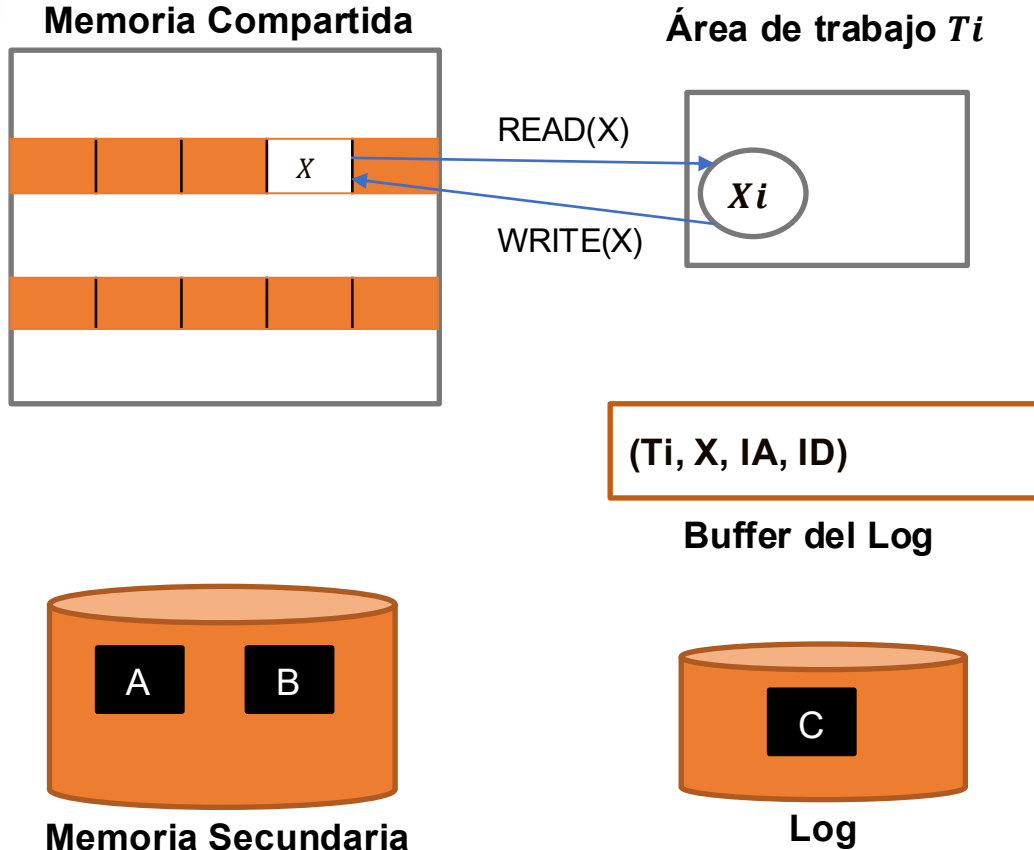
- Cuando se realiza el proceso de restauración
 - Recorrer el Log consume tiempo
 - La mayoría de las transacciones a Rehacer (de acuerdo al algoritmo) ya tienen sus actualizaciones escritas en la BD.
- **¿Cómo hacer para no recorrer el log completo?**

Proceso de recuperación con checkpoints

- **Algunos papers**

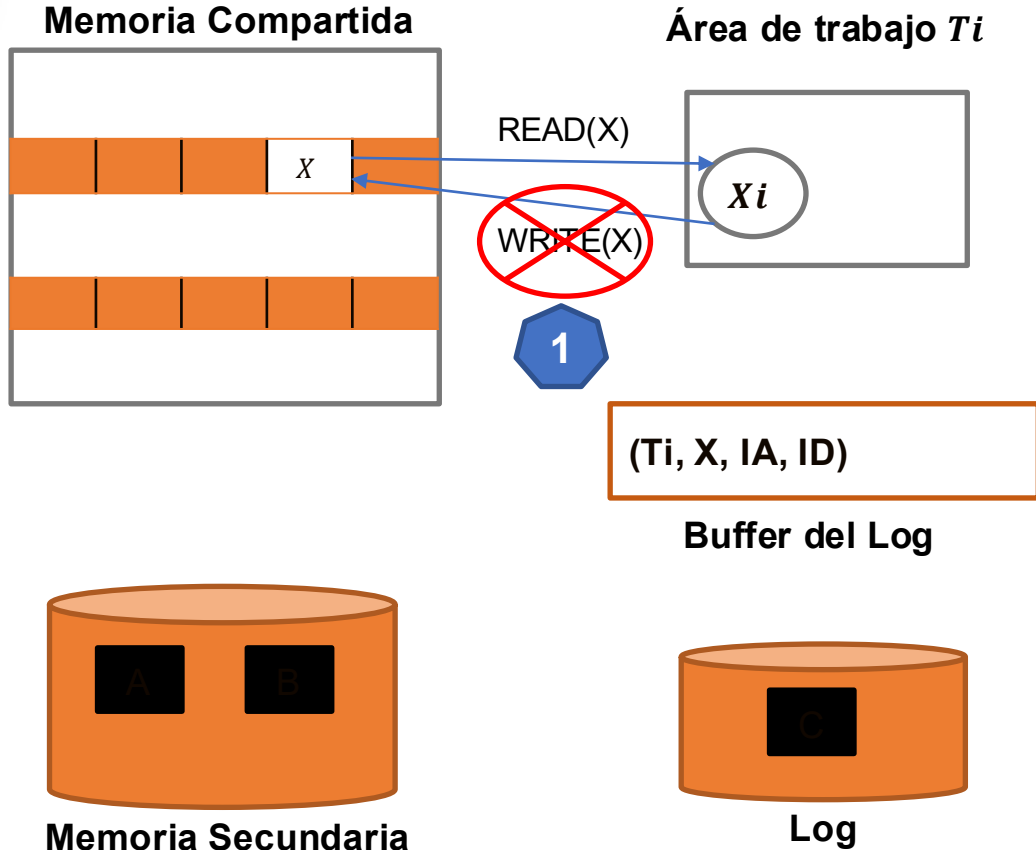
- ✓ Fukumoto, S., Kaio, N., & Osaki, S. (1992). ***A study of checkpoint generations for a database recovery mechanism***. *Computers & Mathematics with applications*, 24(1-2), 63-70.
- ✓ Gray, J., McJones, P. R., Blasgen, M. W., Lindsay, B. G., Lorie, R. A., Price, T. G., ... & Traiger, I. L. (1981). ***The recovery manager of the System R database manager***. *ACM Computing surveys*, 13(2), 223-242.
- ✓ Haerder, T., & Reuter, A. (1983). ***Principles of transaction-oriented database recovery***. *ACM computing surveys (CSUR)*, 15(4), 287-317.

Proceso de recuperación con checkpoints



**Proceso de
checkpoint**

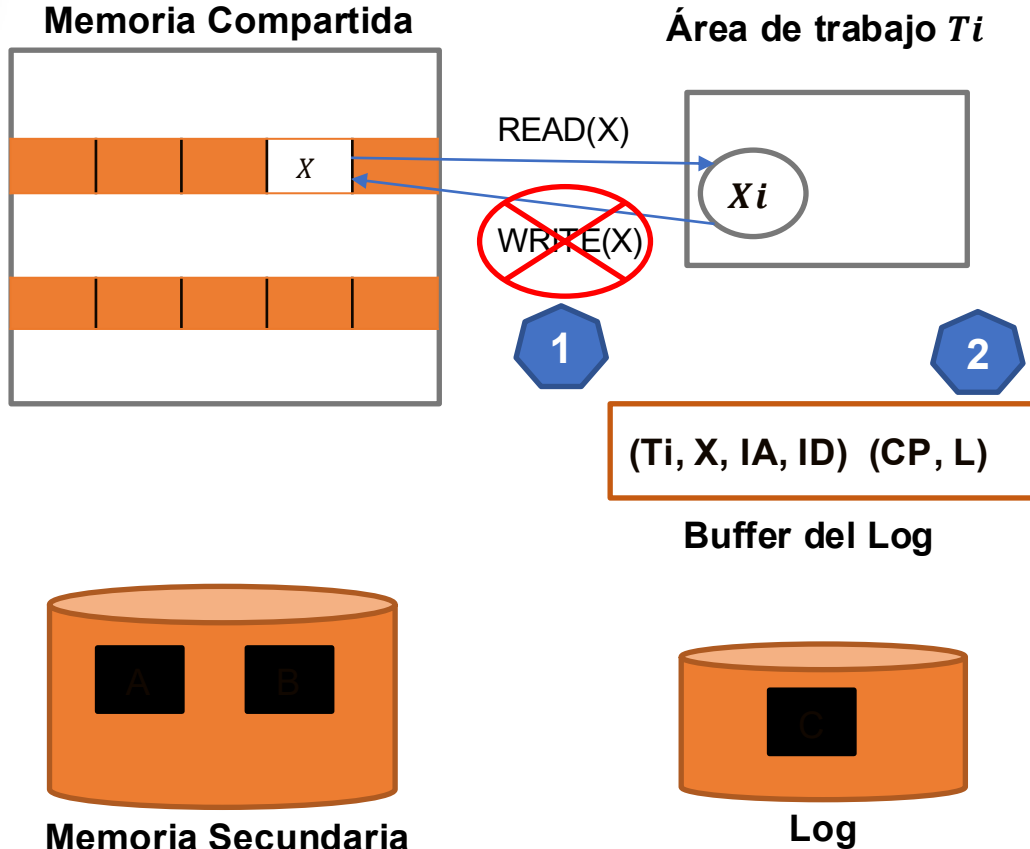
Proceso de recuperación con checkpoints



1

- Se impide que las transacciones realicen actualizaciones.

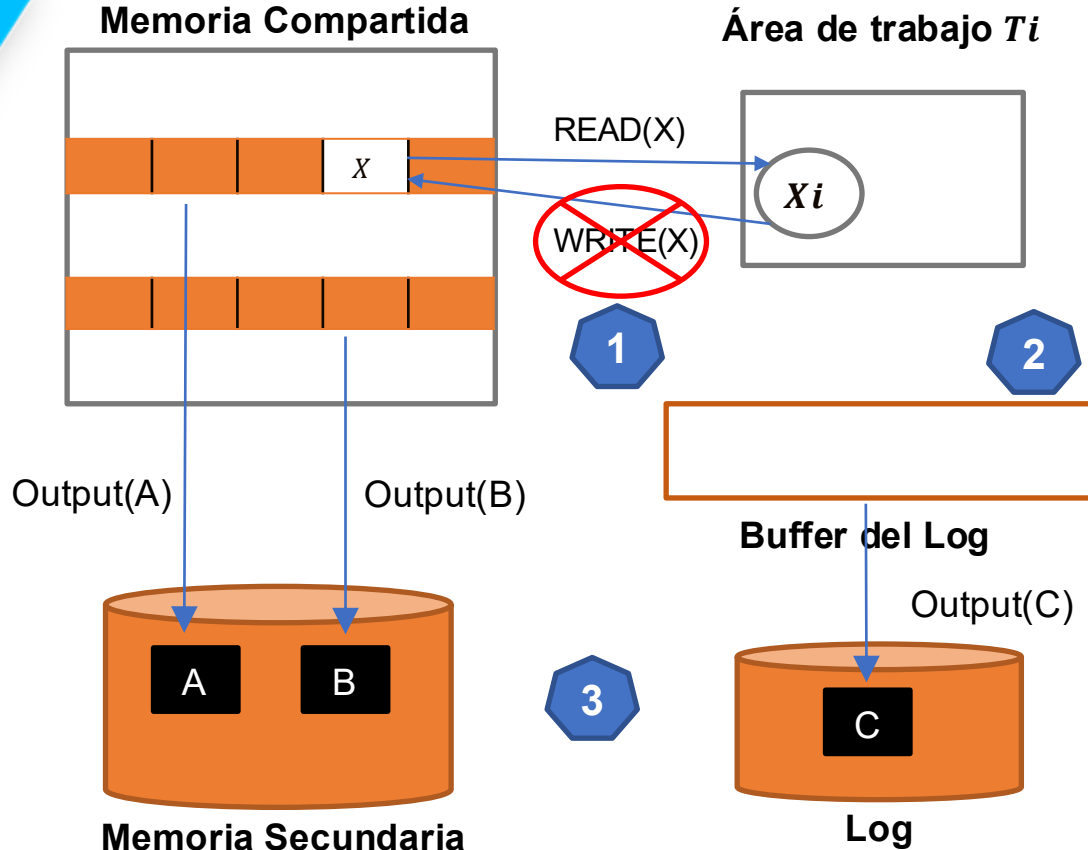
Proceso de recuperación con checkpoints



2

- Se crea un registro en el Log con la lista de transacciones activas al momento del Check Point.

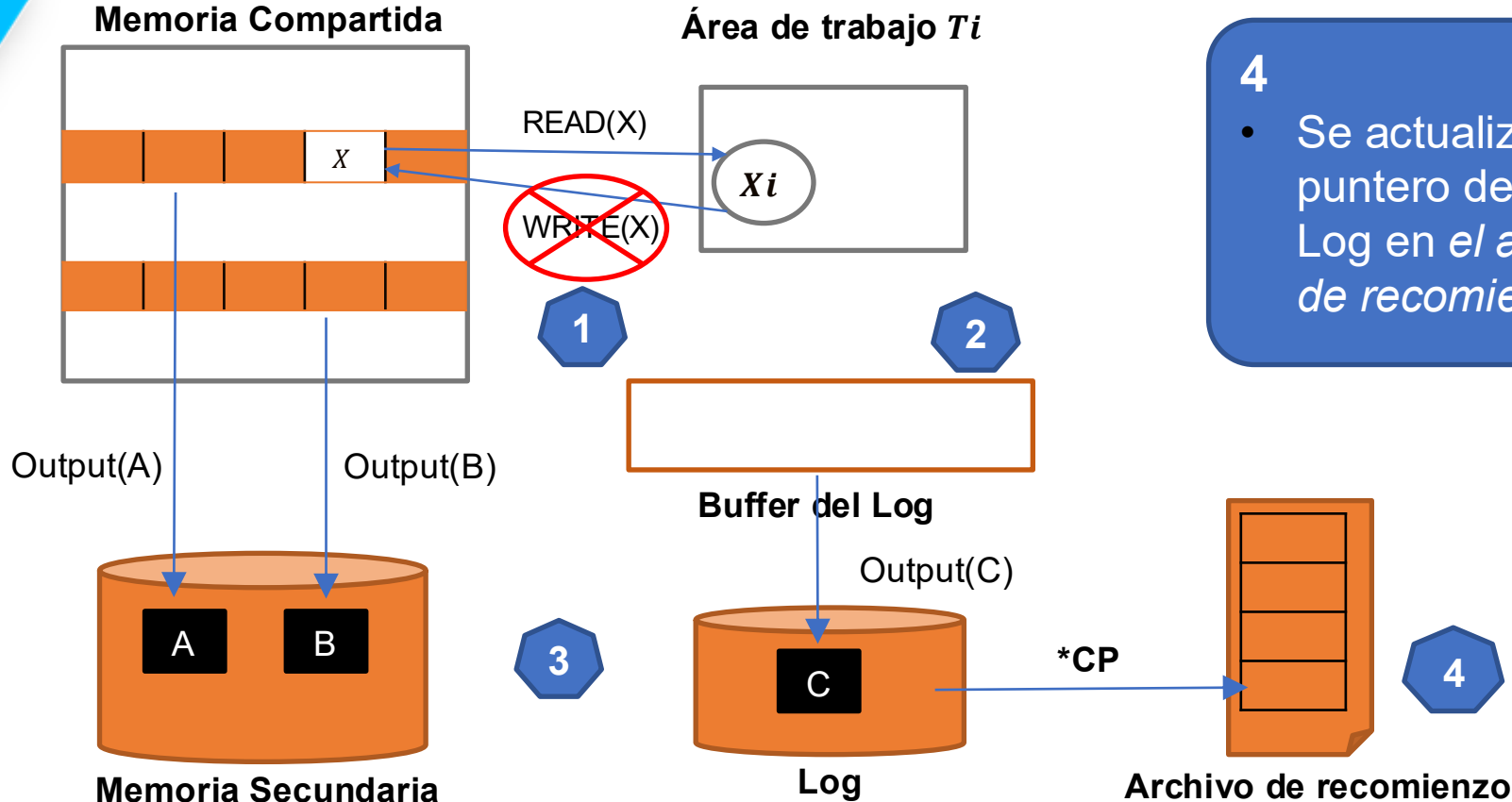
Proceso de recuperación con checkpoints



3

- Se fuerza la salida de los buffers de datos y del Log.

Proceso de recuperación con checkpoints

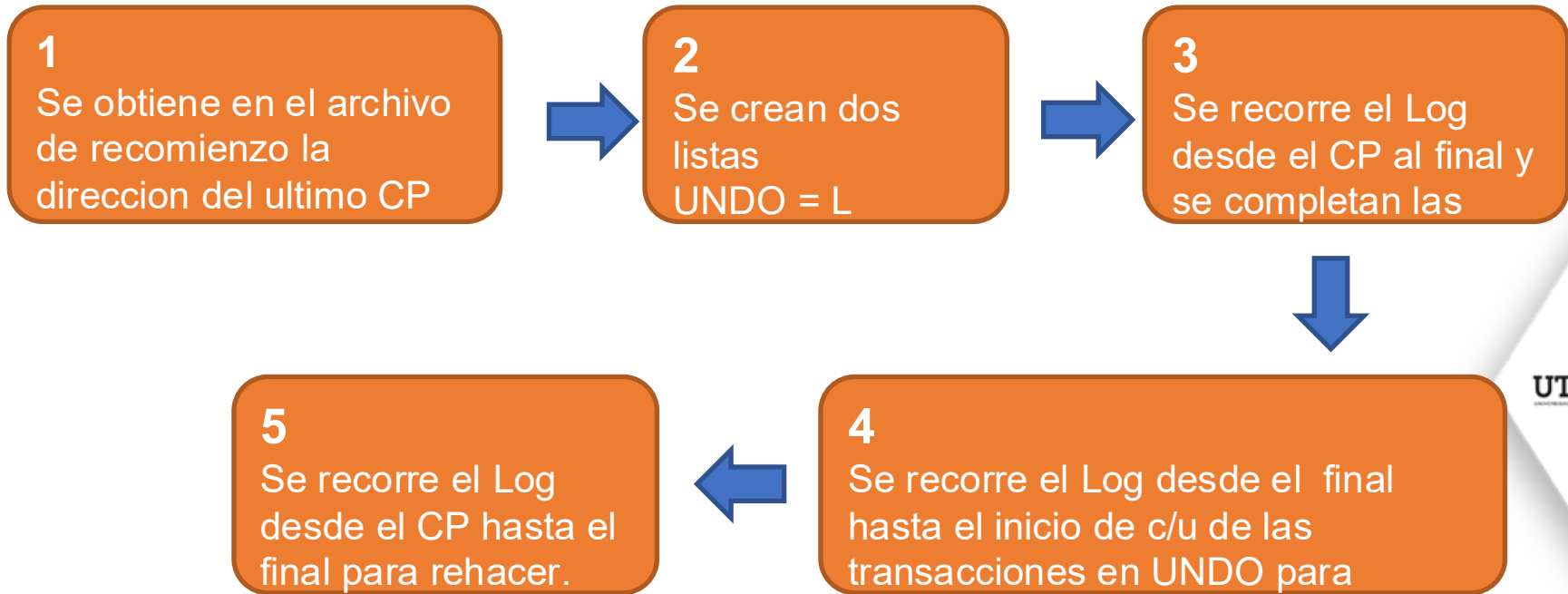


4

- Se actualiza el puntero del CP del Log en el *archivo de recomienzo*.

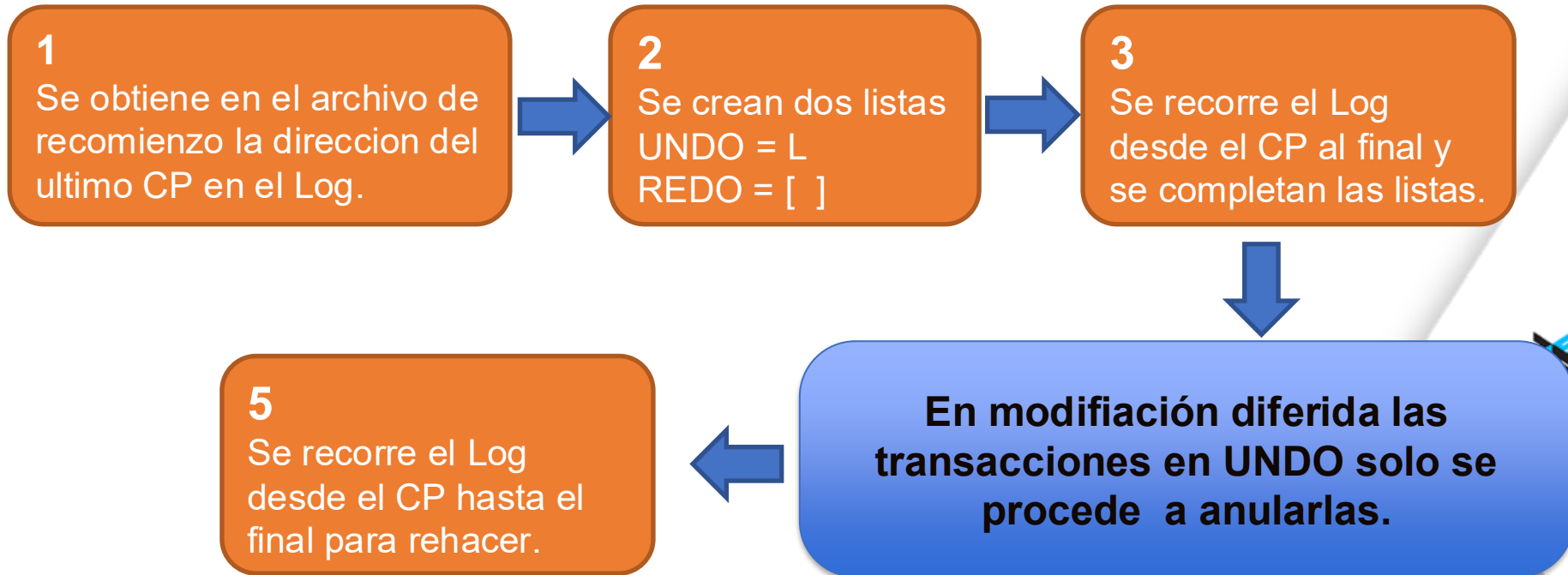
Proceso de recuperación con checkpoints

- Proceso de recuperación ante una falla



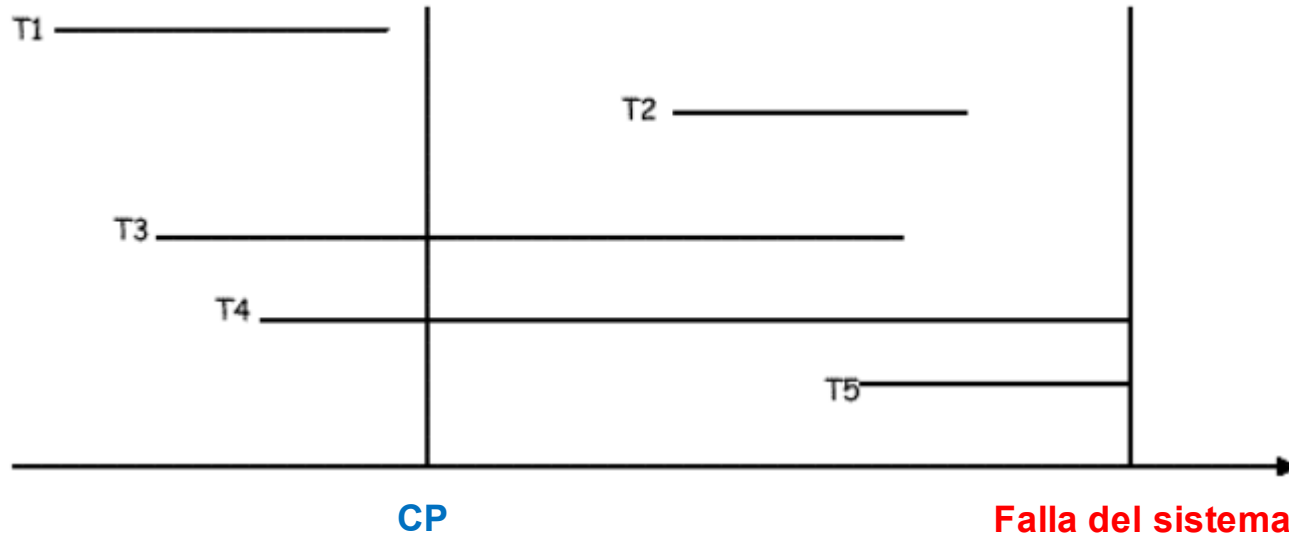
Proceso de recuperación con checkpoints

- Proceso de recuperación ante una falla



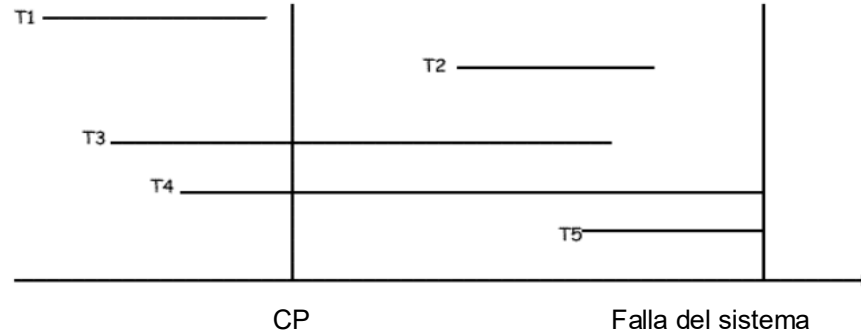
Proceso de recuperación con checkpoints

- Ejemplo de recuperación:



Proceso de recuperación con checkpoints

- Ejemplo de recuperación:



1.
(CP, [T3,
T4])

2.
UNDO= [T3,
T4]
REDO=[]

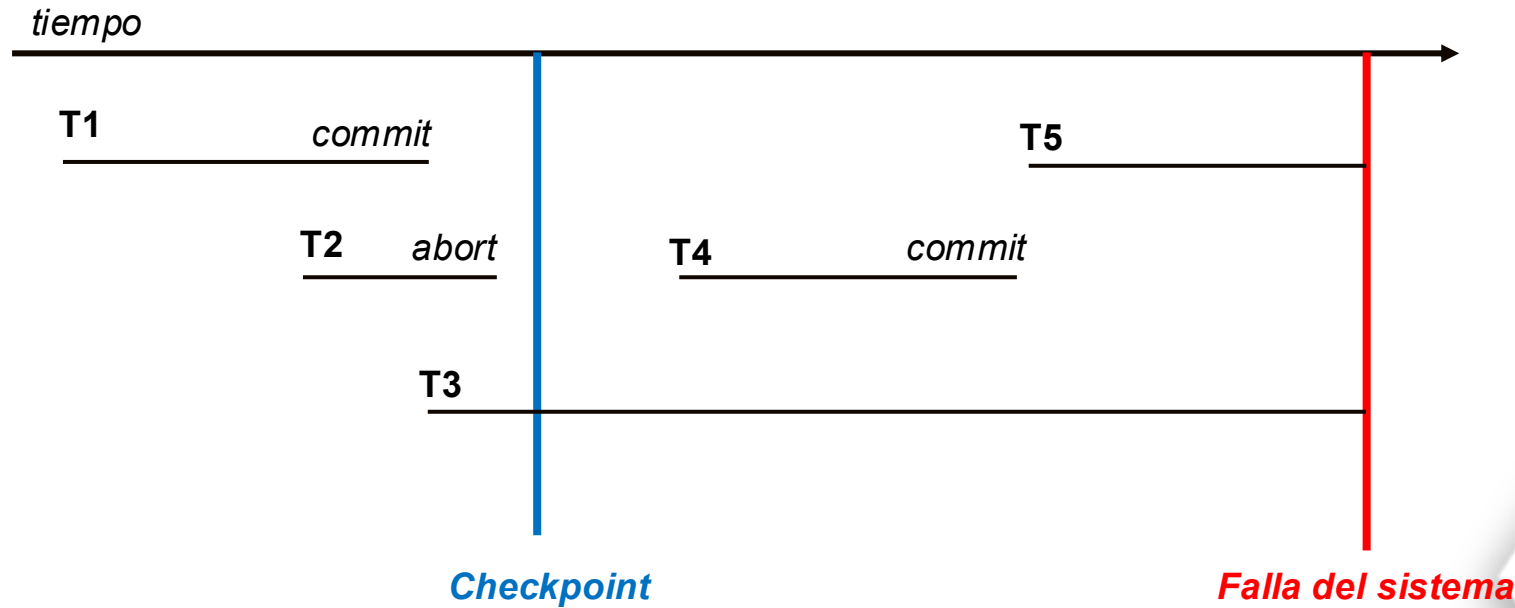
3.
UNDO= [~~T3~~, T4,
T5]
REDO=[T2, T3]

4.
Deshacer
UNDO

5.
Rehacer REDO

No es necesario
hacer REDO a
T1 pues esto
ocurrió antes
del último
checkpoint

Ejercicio



1.
(CP, [T3])

2.
UNDO= [T3]
REDO=[]

3.
UNDO= [T3,T5]
REDO=[T4]

4.
Deshacer UNDO

5.
Rehacer REDO

No es necesario hacer REDO a T1 pues esto ocurrió antes del último checkpoint

Recuperación en fallas de almacenamiento

- **Backups**

- Copias de la BD completa o parcial hasta una fecha dada.
- El Administrador de BD debe programar respaldos periódicos.
- Para la recuperación se usa el respaldo y el Log para las actualizaciones posteriores.

- **Replicación**

- Mantener una BD en otra localización o nodo, que se actualiza a la par de la original, como consecuencia:
 - Aumenta la disponibilidad de la BD en caso de fallas
 - Aumenta también los costos de almacenamiento y comunicación.

3.



Laboratorios 06

Resolver ejercicios